

Proyecto de Desarrollo de Software — App Móvil de Gestión de Club de Atletismo

Universidad Privada Domingo Savio — Carrera de Ingeniería de Sistemas

Santa Cruz de la Sierra, Bolivia · 2026

Secciones Preliminares

Lista de Tablas

N°	Tabla	Capítulo
1	Actores del sistema	3.3.1
2	CU-01 Iniciar Sesión	3.3.1
3	CU-02 Registrar Atleta	3.3.1
4	CU-03 Gestionar Agenda	3.3.1
5	CU-04 Registrar Asistencia	3.3.1
6	CU-05 Registrar y Consultar Rendimiento	3.3.1
7	CU-06 Publicar Convocatoria	3.3.1
8	HU-01 Registro de cuenta	3.3.2
9	HU-02 Inicio de sesión	3.3.2
10	HU-03 Consultar agenda semanal	3.3.2
11	HU-04 Crear y editar sesión	3.3.2
12	HU-05 Registro de asistencia	3.3.2
13	HU-06 Registro de marcas	3.3.2
14	HU-07 Historial de rendimiento	3.3.2
15	HU-08 Evolución grupal	3.3.2
16	HU-09 Publicar convocatoria	3.3.2
17	HU-10 Registrar resultados	3.3.2
18	HU-11 Notificaciones push	3.3.2
19	HU-12 Gestionar perfil atleta	3.3.2
20	HU-13 Editar datos propios	3.3.2
21	HU-14 Gestionar disciplinas	3.3.2
22	HU-15 Registrar condición física	3.3.2
23	HU-16 Vista diferenciada rol PADRE	3.3.2
24	Requisitos Funcionales RF-01 a RF-20	3.2.1
25	Requisitos No Funcionales RNF-01 a RNF-06	3.2.2
26	Entidades del dominio (14 clases)	3.3.3
27	Atributos clave por entidad	3.3.3
28	Capas de arquitectura del software	4.1
29	Modelo lógico de base de datos	4.2.1

	(SQL)	
30	Diccionario de datos — registro_rendimiento	4.2.2
31	Módulos del sistema y componentes	4.4
32	Stack tecnológico — Backend	5.1
33	Stack tecnológico — Frontend Android	5.1
34	Sprints de desarrollo Scrum	5.2
35	Pruebas unitarias (PU-01 a PU-21)	5.4
36	Pruebas de integración (PI-01 a PI-16)	5.4
37	Pruebas de aceptación (PA-01 a PA-20)	5.4
38	Cobertura de pruebas por módulo	5.4
39	Cobertura final de requisitos	6.1

Lista de Figuras

Nº	Figura	Capítulo
1	Diagrama de arquitectura de 3 capas	4.1
2	Diagrama de relaciones del dominio (Mermaid ERD)	3.3.3
3	Mockup — Pantalla de Login	4.3.1
4	Mockup — Dashboard Entrenador	4.3.1
5	Mockup — Dashboard Atleta	4.3.1
6	Mockup — Agenda semanal	4.3.1
7	Mockup — Registro de asistencia	4.3.1
8	Mockup — Gráfica de evolución de marcas	4.3.1
9	Árbol de paquetes del proyecto Spring Boot	5.3
10	Flujo de notificación push (diagrama de secuencia)	Anexo A

Índice de Contenidos

- Capítulo 1. Introducción
- 1.1 Introducción y Contexto del Problema
- 1.2 Justificación del Proyecto
- 1.3 Objetivos
- 1.4 Alcance del Proyecto

- Capítulo 2. Marco Teórico y Referencial
- 2.1 Marco Teórico del Atletismo y Gestión Deportiva
- 2.2 Marco Tecnológico
- Capítulo 3. Requisitos y Análisis del Sistema
- 3.1 Metodología de Desarrollo Adoptada
- 3.2 Requisitos
- 3.3 Análisis de Requisitos
- Capítulo 4. Diseño del Software
- 4.1 Arquitectura del Software
- 4.2 Diseño de la Base de Datos
- 4.3 Diseño de la Interfaz de Usuario
- 4.4 Diseño de Componentes y Módulos
- Capítulo 5. Implementación y Pruebas
- 5.1 Entorno de Implementación
- 5.2 Proceso de Desarrollo
- 5.3 Estructura del Proyecto
- 5.4 Pruebas y Validación
- Capítulo 6. Conclusiones y Trabajo Futuro
- Referencias Bibliográficas
- Anexo A — Registro de Implementación y Cambios
- Anexo B — Fragmentos de Código Fuente Representativos
- Anexo C — Entrevista de Levantamiento de Requisitos
- Anexo E — Glosario de Términos

Capítulo 1. Introducción

1.1 Introducción y Contexto del Problema

El Club Atlético Santa Cruz de la Sierra es una organización deportiva que agrupa aproximadamente 45 atletas distribuidos en cuatro categorías etarias: Pre-Infantil (8-10 años), Infantil (11-13), Juvenil (14-17) y Mayores (+18). Las disciplinas que se practican son velocidad (100m y 200m), salto largo, lanzamiento de bala y gimnasia artística, organizadas en cuatro grupos de entrenamiento según disciplina y entrenador asignado.

En su estado actual, la gestión operativa del club presenta serias deficiencias que afectan a todos sus actores:

- **Comunicación informal:** los horarios de entrenamiento se publican mediante grupos de WhatsApp o fotografías de hojas escritas a mano. Los cambios de último momento frecuentemente no llegan a tiempo a todos los atletas, resultando en asistencia a sesiones canceladas o ausencias injustificadas por falta de información.
- **Registro de asistencia en papel:** el entrenador lleva planillas físicas que eventualmente se transcriben a Excel, proceso en el que se pierde información histórica y se acumula trabajo manual.
- **Historial de rendimiento fragmentado:** las marcas y tiempos se anotan en libretas del entrenador o se envían por WhatsApp. No existe un repositorio histórico accesible para el atleta ni para el análisis de progresión.
- **Gestión de competencias informal:** las inscripciones se realizan por mensaje privado al entrenador, sin confirmación formal. Los resultados se registran en fotografías del cronómetro guardadas en el teléfono, sin vinculación al historial del atleta.

- **Protección de datos insuficiente:** el club tiene atletas menores de edad cuyos datos circulan en grupos de WhatsApp sin control de acceso.

Esta situación genera pérdida de tiempo, frustración en los atletas y decisiones de entrenamiento basadas en información incompleta. La digitalización de estos procesos mediante una aplicación móvil específica para el dominio del atletismo representa una mejora directa y medible en la operación del club.

1.2 Justificación del Proyecto

El proyecto se justifica desde tres dimensiones:

Dimensión operativa: la automatización de la agenda, asistencia y comunicación elimina el trabajo manual repetitivo del entrenador y garantiza que todos los atletas reciban la misma información en tiempo real, independientemente de si leen o no el grupo de WhatsApp.

Dimensión deportiva: contar con un historial digital de marcas permite al entrenador identificar tendencias de rendimiento, planificar entrenamientos específicos por atleta y motivar a los deportistas mostrándoles su evolución. La detección automática de récords personales agrega valor inmediato al proceso de entrenamiento.

Dimensión académica: el proyecto aplica en un contexto real los conceptos de ingeniería de software cubiertos en la carrera: análisis y especificación de requisitos, diseño de arquitecturas en capas, modelado de dominio, desarrollo ágil (Scrum), integración de APIs externas (Firebase Cloud Messaging), despliegue con contenedores Docker y CI/CD.

No existe actualmente ninguna solución comercial de gestión deportiva en Bolivia adaptada a la escala y las necesidades específicas de un club de atletismo amateur. Las aplicaciones internacionales disponibles (TeamApp, TeamSnap) están en inglés, tienen costos de suscripción

elevados para el contexto local y no contemplan las particularidades del atletismo boliviano (categorías, disciplinas, calendario de competencias).

1.3 Objetivos

1.3.1 Objetivo General

Desarrollar una aplicación móvil nativa para Android que automatice y centralice la gestión de entrenamientos, rendimiento deportivo y comunicación del Club Atlético Santa Cruz de la Sierra, mejorando la eficiencia operativa y la experiencia de atletas, entrenadores y padres de familia.

1.3.2 Objetivos Específicos

1. Implementar un sistema de autenticación seguro con roles diferenciados (Administrador, Entrenador, Atleta, Padre/Tutor), verificación de correo electrónico y protección contra ataques de fuerza bruta.
2. Desarrollar un módulo de agenda semanal que permita crear, editar y cancelar sesiones de entrenamiento, con notificación push automática a los atletas del grupo afectado.
3. Digitalizar el registro de asistencia por sesión con marcación de estado (Presente, Ausente, Justificado) y cálculo automático de porcentaje de asistencia.
4. Implementar un historial de marcas deportivas por atleta y disciplina, con detección automática de récord personal y visualización gráfica de evolución individual y grupal.
5. Gestionar el ciclo completo de competencias: publicación de convocatoria, confirmación de participación por atleta y registro de resultados vinculados al historial individual.
6. Integrar notificaciones push en tiempo real vía Firebase Cloud Messaging, con configuración de preferencias por tipo de notificación.

7. Automatizar la actualización de categoría etaria de los atletas mediante un proceso programado diario.
8. Desplegar el backend en un entorno de producción con CI/CD automatizado (GitHub Actions + Coolify) y el APK distribuible mediante GitHub Releases.

1.4 Alcance del Proyecto

Incluido:

Dimensión	Alcance
Plataforma	Android nativo (Java SDK), Android 8.0 (API 26) o superior
Usuarios objetivo	Entrenadores, Atletas, Padres/Tutores del Club Atlético Santa Cruz
Escala	Hasta ~50 usuarios concurrentes (escala del club)
Historias de Usuario	15 HU implementadas (HU-01 a HU-15)
Requisitos Funcionales	20 RF (RF-01 a RF-20)
Módulos	Autenticación, Agenda, Asistencia, Marcas, Competencias, Disciplinas, Condición Física, Notificaciones, Perfiles
Backend	Spring Boot 3.3.6, Java 21, PostgreSQL 16
Despliegue	Coolify en VPS propio, CI/CD con GitHub Actions
Notificaciones	Firebase Cloud Messaging (push a dispositivos Android)

Fuera de alcance:

Exclusión	Justificación
Plataforma iOS	Se requeriría cuenta de Apple Developer (\$99/año) y herramientas adicionales
Modo offline completo	Complejidad de sincronización bidireccional; fuera del alcance del taller
HTTPS / TLS propio	Requiere dominio propio; se aplaza a versión posterior
Publicación en Google Play	Proceso independiente del desarrollo; fuera del alcance académico
Pruebas automatizadas completas	Las pruebas documentadas son especificaciones; se ejecutaron manualmente
Panel web para administración	Alcance exclusivo a la app móvil Android

Capítulo 2. Marco Teórico y Referencial

2.1 Marco Teórico del Atletismo y Gestión Deportiva

2.1.1 El Atletismo como Disciplina Deportiva

El atletismo es considerado el deporte base por excelencia, al combinar en sus distintas especialidades las capacidades físicas fundamentales del ser humano: correr, saltar y lanzar. Según la World Athletics (organismo rector internacional), el atletismo se organiza en cuatro grandes grupos de pruebas: carreras (velocidad, medio fondo, fondo), saltos (longitud, triple, altura, pértiga), lanzamientos (peso, disco, martillo, jabalina) y pruebas combinadas (pentatlón, decatlón). En el contexto del Club Atlético Santa Cruz, las disciplinas activas son velocidad (100m, 200m), salto largo, lanzamiento de bala y gimnasia artística.

2.1.2 Gestión de Clubes Deportivos

La gestión de un club deportivo amateur abarca procesos organizativos que, sin herramientas digitales adecuadas, consumen tiempo desproporcionado al entrenador: planificación de sesiones, control de asistencia, seguimiento del rendimiento individual, coordinación de competencias y comunicación con atletas y familias. Autores como Chelladurai (2014) en *Sport Management: Principles and Applications* identifican la información oportuna como uno de los factores críticos para el desempeño de un club: los atletas que conocen con antelación sus sesiones, sus marcas y su evolución tienen mayor adherencia al entrenamiento y mejores resultados competitivos.

La digitalización de procesos deportivos no es una tendencia nueva en el deporte profesional, pero su penetración en el atletismo amateur latinoamericano es aún incipiente. Aplicaciones como TeamApp, TeamSnap o Sportstrio abordan parcialmente esta necesidad pero presentan

barreras de costo, idioma y configurabilidad que limitan su adopción en clubes locales de Bolivia.

2.1.3 Categorías Etarias en el Atletismo Boliviano

La Federación Boliviana de Atletismo organiza a los competidores en categorías basadas en la edad: Pre-Infantil (8-10 años), Infantil (11-13), Juvenil (14-17) y Mayores (+18). Estas categorías determinan las distancias de competencia, los récords de referencia y las reglas aplicables. En el sistema desarrollado, la categoría de cada atleta se calcula automáticamente a partir de la fecha de nacimiento y se actualiza de forma programada diariamente, garantizando que el dato siempre refleje la categoría vigente del atleta sin intervención manual del entrenador.

2.1.4 Protección de Datos de Menores en Entornos Deportivos

El club cuenta con atletas menores de edad cuyos datos personales (nombre, fecha de nacimiento, foto, información del tutor) deben tratarse con especial cuidado. El sistema implementa control de acceso por rol: los datos de menores son accesibles exclusivamente para el Entrenador, el Administrador y el tutor vinculado al perfil. Ningún otro atleta puede acceder a la información de un menor.

2.2 Marco Tecnológico

2.2.1 Desarrollo Móvil Android Nativo (Java)

El desarrollo nativo en Android implica construir la aplicación directamente con el SDK de Android, usando Java o Kotlin como lenguaje y XML para la definición de interfaces. Este enfoque ofrece acceso completo a las APIs del sistema operativo, mejor rendimiento que los frameworks híbridos y control total sobre el comportamiento de la aplicación. En este proyecto

se utilizó Java 21 (Oracle, 2024), por ser el lenguaje de mayor dominio del equipo de desarrollo y por su amplia documentación oficial.

Material Design 3 es el sistema de diseño de Google para aplicaciones Android (Google, 2023), implementado en este proyecto mediante la biblioteca `com.google.android.material`. Provee componentes visuales (botones, campos de texto, tarjetas, navegación inferior) con soporte nativo para temas oscuros, accesibilidad y animaciones de transición.

2.2.2 Arquitectura REST y Spring Boot

REST (*Representational State Transfer*) es un estilo arquitectónico para sistemas distribuidos que define un conjunto de restricciones sobre cómo los componentes interactúan a través de la red. En una API REST, los recursos se identifican mediante URLs y las operaciones se expresan con métodos HTTP estándar (GET, POST, PUT, DELETE).

Spring Boot es una extensión del ecosistema Spring Framework que elimina la configuración boilerplate mediante autoconfiguración. Permite crear aplicaciones Java standalone con servidor Tomcat embebido que se empaquetan como un JAR ejecutable (Pivotal Software, 2024). En este proyecto se utilizó la versión 3.3.6 con Java 21, aprovechando las mejoras de rendimiento de las `virtual threads` y las expresiones `switch` de Java 17+.

Spring Security 6 gestiona la autenticación y autorización de todos los endpoints. La estrategia es JWT (*JSON Web Token*) sin estado: al autenticarse, el backend emite un token firmado con HMAC-SHA256 que el cliente incluye en cada petición subsiguiente en el header `Authorization: Bearer <token>`. No se mantiene estado de sesión en el servidor.

2.2.3 Persistencia — PostgreSQL y Hibernate ORM

PostgreSQL 16 es el sistema de gestión de bases de datos relacional utilizado en producción (PostgreSQL Global Development Group, 2024). Sus características clave para este proyecto

son: tipos ENUM nativos (para roles, estados de sesión, estados de asistencia), soporte para columnas con DEFAULT en operaciones ALTER TABLE, índices compuestos y restricciones de integridad referencial (ON DELETE RESTRICT).

Hibernate ORM es la implementación de referencia de la especificación JPA (Jakarta Persistence API) (Hibernate ORM, 2024). Mapea automáticamente clases Java anotadas (@Entity) a tablas de base de datos y genera el SQL necesario para las operaciones CRUD. La configuración ddl-auto: update permite que Hibernate evolucione el esquema de forma incremental al detectar nuevas columnas o tablas en las entidades.

2.2.4 Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging es el servicio de Google para el envío de notificaciones push a dispositivos Android e iOS (Firebase, 2024). La arquitectura de FCM es cliente-servidor: el backend (mediante el Firebase Admin SDK) envía un mensaje al servidor de FCM indicando el token del dispositivo destino, y FCM se encarga de la entrega al dispositivo mediante una conexión persistente mantenida por el sistema operativo Android. El token FCM de cada usuario se registra en la base de datos al iniciar sesión en la app.

2.2.5 Contenedores Docker y Despliegue Continuo

Docker es una plataforma de contenedores que empaqueta la aplicación y todas sus dependencias en una imagen portable. El backend del proyecto se empaqueta en una imagen Docker que se construye en GitHub Actions y se despliega automáticamente en Coolify (plataforma open source de orquestación de contenedores autoalojada en VPS).

GitHub Actions es el sistema de CI/CD integrado en GitHub. En este proyecto se configuraron dos workflows: uno que construye el APK de Android en cada push a master y lo publica como artefacto, y otro (manejado por Coolify via webhook) que redespliega el backend en producción

con healthcheck y rollback automático ante fallo. Estas prácticas de despliegue con verificación de salud y reversión automática siguen los principios de software listo para producción descritos por Nygard (2018).

2.2.6 Retrofit 2 y OkHttp3

Retrofit 2 es una librería Android que convierte interfaces Java anotadas en llamadas HTTP (Square Open Source, 2024). Combinado con un convertidor Gson, deserializa automáticamente el JSON de las respuestas a objetos Java. **OkHttp3** es el cliente HTTP subyacente que gestiona la conexión, caché, y los interceptores (en este proyecto: interceptor de autenticación que añade el JWT a cada petición, e interceptor de 401 que redirige al Login al detectar sesión expirada).

2.2.7 MPAndroidChart

MPAndroidChart es una librería open source para Android que provee componentes de visualización de datos: gráficas de línea, barras, torta, radar, entre otras. En este proyecto se utiliza `LineChart` para las gráficas de evolución de marcas individuales (modo Cubic Bezier, relleno bajo la curva) y la comparativa multi-línea de evolución grupal (múltiples `LineDataSet` con paleta de colores diferenciada por atleta).

Capítulo 3. Requisitos y Análisis del Sistema

3.1 Metodología de Desarrollo Adoptada

El proyecto adoptó **Scrum** (Schwaber & Sutherland, 2020) como marco de trabajo ágil, alineado con los valores del *Manifiesto Ágil* (Agile Alliance, 2001) y adaptado a un equipo unipersonal con sprints de dos semanas. Las principales razones de esta elección son:

- **Entrega incremental de valor:** cada sprint produce funcionalidad funcionando y desplegada, lo que permite verificar con el cliente (el propio club) que el software resuelve el problema correcto.
- **Gestión de cambios:** durante el desarrollo, algunos requisitos cambiaron (cambio de React Native a Android nativo, incorporación de CI/CD, simplificación de módulos). Scrum absorbió estos cambios sin romper el proceso.
- **Priorización basada en impacto:** el backlog permite reordenar historias según el valor entregado al usuario, asegurando que las funcionalidades más críticas (autenticación, agenda, notificaciones) se construyan primero.

Artefactos utilizados:

Artefacto	Implementación
Product Backlog	13 Historias de Usuario priorizadas por valor de negocio
Sprint Backlog	Selección de HU y tareas técnicas para cada sprint de 2 semanas
Incremento	APK funcional + backend desplegado al final de cada sprint
Definition of Done	HU implementada, API endpoint funcionando, probada en dispositivo físico

Convenciones técnicas adoptadas:

- Control de versiones: Git, rama única master con commits por funcionalidad
- Estilo de código Java: Google Java Style Guide
- Arquitectura estricta: Controller → Service → Repository (sin saltar capas)
- DTOs en toda comunicación entre capas (las entidades JPA nunca se exponen en endpoints)
- Validación: Bean Validation en todos los DTOs de entrada
- Manejo de errores: GlobalExceptionHandler con @RestControllerAdvice para respuestas JSON uniformes

3.2 Requisitos

3.2.1 Requisitos Funcionales

RF — Gestión de Usuarios y Autenticación

ID	Nombre	Descripción	Prioridad
RF-01	Registro de usuario	El sistema debe permitir registrar nuevos usuarios con nombre, correo, contraseña y rol (Administrador, Entrenador, Atleta, Padre/Tutor). Los menores deben vincularse a un tutor.	Alta
RF-02	Autenticación con roles	El sistema debe autenticar usuarios y redirigirlos a vistas diferenciadas según su rol. Las contraseñas deben almacenarse con hash seguro.	Alta
RF-03	Recuperación de contraseña	El sistema debe permitir recuperar la contraseña mediante correo electrónico con enlace válido por 24 horas.	Alta
RF-04	Gestión de perfiles de atletas	El entrenador debe poder crear, editar, desactivar y consultar el perfil completo de cada atleta, incluyendo foto, categoría, disciplina y datos del tutor.	Alta

RF — Gestión de Agenda y Entrenamientos

ID	Nombre	Descripción	Prioridad
RF-05	Crear y editar sesiones	El entrenador debe poder crear sesiones indicando: fecha, hora inicio/fin, lugar, grupo asignado y descripción. Debe poder editarlas o cancelarlas.	Alta
RF-06	Consultar agenda semanal	Los atletas deben poder visualizar la agenda semanal con navegación por semanas. Las sesiones canceladas deben	Alta

		mostrarse con etiqueta visible.	
RF-07	Registro de asistencia	El entrenador debe poder registrar la asistencia por sesión marcando a cada atleta como Presente, Ausente o Justificado. El sistema debe calcular el porcentaje.	Alta
RF-08	Consultar historial de asistencia	El entrenador puede ver el historial de asistencia por atleta y por sesión. El atleta puede ver su propio historial.	Media

RF — Seguimiento del Rendimiento

ID	Nombre	Descripción	Prioridad
RF-09	Registrar marcas y resultados	El entrenador debe poder registrar resultados por atleta indicando: disciplina, fecha, valor de la marca y contexto (entrenamiento o competencia).	Alta
RF-10	Consultar historial de rendimiento	El atleta debe poder consultar su historial de marcas por disciplina, ordenado cronológicamente, con gráfica de evolución.	Alta
RF-11	Ver evolución grupal	El entrenador debe poder comparar el rendimiento de todos los atletas de su grupo por disciplina, con indicadores de mejora o retroceso.	Media
RF-12	Detectar marca personal	El sistema debe identificar automáticamente cuándo un nuevo resultado supera la marca personal del atleta y registrarlo con distinción visual.	Media

RF — Gestión de Competencias

ID	Nombre	Descripción	Prioridad
RF-13	Publicar convocatorias	El entrenador debe poder crear convocatorias con:	Media

		nombre del evento, fecha, lugar, disciplinas y atletas/grupos convocados.	
RF-14	Confirmación de participación	Los atletas convocados deben poder confirmar o declinar su participación desde la app.	Media
RF-15	Registrar resultados de competencia	El entrenador debe poder ingresar los resultados de cada atleta asociados a la competencia correspondiente.	Media

RF — Notificaciones y Comunicación

ID	Nombre	Descripción	Prioridad
RF-16	Notificaciones push automáticas	El sistema debe enviar notificaciones push automáticas ante: cancelación de sesión, cambio de horario, nueva convocatoria y publicación de resultados.	Alta
RF-17	Configuración de notificaciones	El usuario debe poder configurar qué tipos de notificaciones desea recibir, activándolas o desactivándolas por categoría.	Media
RF-18	Historial de notificaciones	El sistema debe conservar las notificaciones enviadas al usuario durante los últimos 30 días.	Baja
RF-19	Gestión de disciplinas	El entrenador debe poder crear, editar y activar/desactivar disciplinas deportivas con requisitos físicos mínimos (peso, altura, IMC, masa muscular, % grasa). Las disciplinas activas deben estar disponibles en todos los selectores de la app.	Alta
RF-20	Registro de condición física	El entrenador debe poder registrar mediciones físicas de cada atleta (peso, altura, masa muscular, % grasa). El IMC se calcula automáticamente. El atleta puede consultar su	Alta

		propio historial.	
--	--	-------------------	--

3.2.2 Requisitos No Funcionales

ID	Categoría	Requisitos
RNF-01	Rendimiento	La app debe cargar la pantalla principal en < 3 segundos con 3G. La agenda semanal en < 2 segundos. Las notificaciones push en < 60 segundos. El sistema debe soportar al menos 200 usuarios simultáneos.
RNF-02	Seguridad	Contraseñas con hash bcrypt. Sesiones con JWT con expiración. Datos de menores protegidos. Comunicaciones sobre HTTPS/TLS. Bloqueo tras 5 intentos fallidos.
RNF-03	Usabilidad	Utilizable por personas con conocimiento básico en < 10 min. Elementos táctiles mínimo 44×44 pt. Compatible con Android 8.0+. Mensajes de error en español.
RNF-04	Disponibilidad offline	Disponibilidad del 99% mensual. App muestra última agenda cacheada sin conexión. Asistencia registrable offline con sincronización posterior.
RNF-05	Mantenibilidad	Arquitectura en capas documentada. Historial conservado mínimo 3 años. Actualizaciones sin migración manual. Logs de auditoría de operaciones críticas.
RNF-06	Portabilidad	Disponible en Google Play y App Store. Backend deployable en Firebase/AWS/Railway. Exportaciones en PDF y Excel legibles en cualquier dispositivo.

3.3 Análisis de Requisitos

3.3.1 Casos de Uso

Actores del sistema

Actor	Tipo	Descripción
Administrador	Principal	Gestiona el sistema completo: usuarios, permisos, configuración general del club.
Entrenador	Principal	Gestiona agenda, asistencia, rendimiento y competencias de su grupo de atletas.
Atleta	Principal	Consulta su agenda, historial de rendimiento y recibe notificaciones.
Padre / Tutor	Secundario	Consulta la agenda y el rendimiento de su hijo menor. Recibe notificaciones relacionadas.

CU-01 — Iniciar Sesión

Campo	Detalle
Actores	Entrenador, Atleta, Padre/Tutor
Precondición	El usuario tiene una cuenta registrada y verificada.
Postcondición	El usuario accede a la vista correspondiente a su rol.
RF relacionados	RF-01, RF-02
RNF relacionados	RNF-02 (Seguridad), RNF-01 (Rendimiento)

Flujo principal:

9. El usuario abre la aplicación móvil.
10. El sistema muestra la pantalla de inicio de sesión.
11. El usuario ingresa su correo electrónico y contraseña.
12. El usuario presiona el botón "Iniciar Sesión".
13. El sistema valida las credenciales contra la base de datos.
14. El sistema genera un token de sesión (JWT) y redirige al usuario a su pantalla de inicio según su rol.

Flujos alternos:

- FA-01: Si las credenciales son incorrectas, el sistema muestra "Correo o contraseña incorrectos" sin especificar cuál falló. Vuelve al paso 2.

- FA-02: Si el usuario falló 5 veces, el sistema bloquea la cuenta 15 minutos y muestra aviso.
- FA-03: Si el usuario presiona "¿Olvidaste tu contraseña?", el sistema inicia el flujo de recuperación.

CU-02 — Registrar Atleta

Campo	Detalle
Actores	Entrenador / Administrador
Precondición	El actor ha iniciado sesión con rol Entrenador o Administrador.
Postcondición	El perfil del atleta queda registrado en el sistema.
RF relacionados	RF-04
RNF relacionados	RNF-02 (Seguridad - privacidad de menores), RNF-03 (Usabilidad)

Flujo principal:

15. El entrenador accede al módulo "Atletas" desde el menú principal.
16. Presiona el botón "Nuevo Atleta".
17. El sistema muestra el formulario de registro.
18. El entrenador ingresa: nombre completo, fecha de nacimiento, disciplina, categoría, grupo y (si es menor) datos del tutor.
19. El entrenador sube la foto del atleta (opcional).
20. El entrenador presiona "Guardar".
21. El sistema valida los datos, genera un ID único para el atleta y guarda el perfil.
22. El sistema muestra confirmación: "Atleta registrado correctamente".

Flujos alternos:

- FA-01: Si el nombre ya existe en el club, el sistema muestra advertencia.
- FA-02: Si faltan campos obligatorios, el sistema resalta los campos en rojo y no permite guardar.

- FA-03: Si el atleta es menor y no se ingresan datos del tutor, el sistema solicita confirmación.

CU-03 — Gestionar Agenda de Entrenamientos

Campo	Detalle
Actores	Entrenador (principal), Atleta (consulta)
Precondición	El entrenador ha iniciado sesión. Existen grupos de atletas registrados.
Postcondición	La sesión queda registrada/modificada y los atletas son notificados.
RF relacionados	RF-05, RF-06, RF-16
RNF relacionados	RNF-01 (Rendimiento), RNF-04 (Disponibilidad offline)

Flujo principal:

23. El entrenador accede al módulo "Agenda".
24. Selecciona la semana deseada en el calendario.
25. Presiona "Nueva Sesión".
26. El sistema muestra el formulario: fecha, hora inicio/fin, lugar, grupo, descripción.
27. El entrenador completa los campos y presiona "Guardar".
28. El sistema valida que no exista conflicto de horario para el mismo grupo.
29. La sesión se registra y el sistema envía notificación push al grupo asignado.

Flujos alternos:

- FA-01 (Cancelar sesión): El entrenador selecciona una sesión existente, elige "Cancelar" e ingresa el motivo. El sistema actualiza el estado y notifica a los atletas.
- FA-02 (Editar sesión): Si cambia la hora, el sistema re-notifica al grupo.
- FA-03 (Conflicto de horario): El sistema muestra advertencia.

CU-04 — Registrar Asistencia

Campo	Detalle
Actores	Entrenador
Precondición	Existe al menos una sesión programada para el día.

Postcondición	La asistencia queda registrada y el sistema actualiza el porcentaje de cada atleta.
RF relacionados	RF-07, RF-08
RNF relacionados	RNF-04 (Disponibilidad offline), RNF-03 (Usabilidad en campo)

Flujo principal:

30. El entrenador accede al módulo "Asistencia" o desde la sesión del día.
31. El sistema muestra la lista de atletas del grupo asignado.
32. El entrenador marca a cada atleta como: Presente, Ausente o Justificado.
33. El entrenador presiona "Guardar Asistencia".
34. El sistema registra la asistencia, calcula el % del grupo y actualiza el historial individual.
35. El sistema muestra el resumen: X presentes, Y ausentes, Z justificados.

Flujos alternos:

- FA-01 (Sin conexión): El registro se guarda localmente y se sincroniza al recuperar conexión.
- FA-02 (Registro tardío): El entrenador puede registrar asistencia hasta 2 horas después de finalizada la sesión.
- FA-03 (Corrección posterior): Solo el Administrador puede modificar un registro ya guardado.

CU-05 — Registrar y Consultar Rendimiento

Campo	Detalle
Actores	Entrenador (registro), Atleta (consulta)
Precondición	El atleta tiene un perfil registrado.
Postcondición	La marca queda registrada en el historial del atleta.
RF relacionados	RF-09, RF-10, RF-11, RF-12
RNF relacionados	RNF-02 (Seguridad), RNF-01 (Rendimiento gráficas)

Flujo principal:

36. El entrenador accede al módulo "Rendimiento".
37. Selecciona al atleta y la disciplina.
38. Ingresa la fecha, el valor de la marca y el contexto (entrenamiento o competencia).
39. Presiona "Registrar".
40. El sistema compara el resultado con el historial del atleta.
41. Si el valor supera la marca personal anterior, el sistema lo marca automáticamente como "Marca Personal".
42. El registro queda guardado y visible en el historial del atleta.
43. El atleta puede acceder a su historial y ver la gráfica de evolución.

CU-06 — Publicar Convocatoria a Competencia

Campo	Detalle
Actores	Entrenador
Precondición	Existen atletas registrados en el sistema.
Postcondición	La convocatoria queda publicada y los atletas convocados son notificados.
RF relacionados	RF-13, RF-14, RF-15, RF-16
RNF relacionados	RNF-01 (Rendimiento notificaciones), RNF-03 (Usabilidad)

Flujo principal:

44. El entrenador accede al módulo "Competencias".
45. Presiona "Nueva Competencia".
46. Completa el formulario: nombre, fecha, lugar, disciplinas y grupos/atletas convocados.
47. Presiona "Publicar".
48. El sistema guarda la competencia y envía notificación push a los convocados.
49. Los atletas reciben la notificación y pueden confirmar o declinar participación.
50. El entrenador visualiza en tiempo real el estado de confirmaciones.

3.3.2 Historias de Usuario

Formato: Como [rol], quiero [funcionalidad] para [beneficio].

Módulo M1 — Autenticación y Gestión de Usuarios

HU-01 — Registro de cuenta de usuario

Campo	Detalle
Rol	Atleta / Padre de atleta menor
Historia	Como Atleta/Padre, quiero registrarme con mi correo y contraseña en la app para tener acceso seguro a mis datos y los de mi hijo.
Prioridad	ALTA

Criterios de aceptación:

- El sistema permite registrarse con nombre completo, correo electrónico y contraseña.
- La contraseña debe tener mínimo 8 caracteres, una mayúscula y un número.
- El sistema envía un correo de verificación antes de activar la cuenta.
- Si el correo ya está registrado, el sistema muestra un mensaje de error claro.
- Para atletas menores, el tutor puede vincular el perfil del menor a su cuenta.

Criterios de calidad:

- **Seguridad:** Contraseñas almacenadas con hash (bcrypt o equivalente).
- **Usabilidad:** Formulario completado en menos de 2 minutos.

HU-02 — Inicio de sesión

Campo	Detalle
Rol	Entrenador / Atleta / Padre
Historia	Como Entrenador/Atleta/Padre, quiero iniciar sesión con mi correo y contraseña para acceder a las funcionalidades correspondientes a mi rol.
Prioridad	ALTA

Criterios de aceptación:

- El sistema autentica al usuario con correo y contraseña.
- Si las credenciales son incorrectas, muestra mensaje de error sin revelar cuál campo falló.
- Después de 5 intentos fallidos, bloquea la cuenta temporalmente por 15 minutos.
- El sistema recuerda la sesión del usuario por 30 días si marca "Recordarme".
- Cada rol redirige a una pantalla de inicio diferente.

Criterios de calidad:

- **Seguridad:** Uso de JWT para manejo de sesiones.
- **Rendimiento:** Inicio de sesión en menos de 3 segundos con conexión normal.

Módulo M2 — Gestión de Agenda y Entrenamientos

HU-03 — Consultar agenda semanal de entrenamientos

Campo	Detalle
Rol	Atleta
Historia	Como Atleta, quiero ver la agenda semanal de entrenamientos en la app para saber con anticipación cuándo y dónde es el próximo entrenamiento sin depender de WhatsApp.
Prioridad	ALTA

Criterios de aceptación:

- La agenda muestra los entrenamientos de la semana actual con fecha, hora, lugar y grupo.
- El atleta puede navegar hacia la semana siguiente o anterior.
- Los entrenamientos cancelados aparecen tachados con etiqueta "CANCELADO".
- Si no hay entrenamientos, muestra "Sin entrenamientos esta semana".

HU-04 — Crear y editar sesión de entrenamiento

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero crear, editar y cancelar sesiones de entrenamiento en la agenda para mantener

	a todos los atletas informados de forma centralizada.
Prioridad	ALTA

Criterios de aceptación:

- El entrenador puede crear una sesión con: fecha, hora inicio/fin, lugar, grupo y descripción.
- Puede editar cualquier campo de una sesión existente.
- Puede cancelar una sesión indicando el motivo.
- Al guardar cualquier cambio, el sistema envía notificación push a los atletas del grupo.
- No se pueden crear dos sesiones con conflicto de horario para el mismo grupo.

HU-05 — Registro de asistencia

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero registrar la asistencia de los atletas a cada sesión de entrenamiento para tener un historial digital sin depender de planillas en papel.
Prioridad	ALTA

Criterios de aceptación:

- El entrenador accede a la lista de atletas del grupo desde la sesión del día.
- Puede marcar a cada atleta como: Presente, Ausente o Justificado.
- La asistencia se puede registrar hasta 2 horas después de finalizada la sesión.
- Una vez guardada, la asistencia solo puede modificarla un Administrador.

Módulo M3 — Seguimiento del Rendimiento

HU-06 — Registro de marcas y resultados

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero registrar las marcas y tiempos obtenidos por cada atleta para contar con un historial digital preciso.

Prioridad	ALTA
------------------	------

Criterios de aceptación:

- El entrenador selecciona al atleta, la disciplina, la fecha y registra el resultado.
- El sistema admite registros para: 100m, 200m, 400m, salto largo, lanzamiento de bala y gimnasia.
- Si el resultado es el mejor del atleta, el sistema lo marca automáticamente como "Marca Personal".
- Los registros no pueden ser modificados por el propio atleta.

HU-07 — Consultar historial de rendimiento propio

Campo	Detalle
Rol	Atleta
Historia	Como Atleta, quiero ver mi historial de marcas y tiempos a lo largo del tiempo para monitorear mi progreso personal.
Prioridad	ALTA

Criterios de aceptación:

- El atleta visualiza su historial ordenado por fecha (más reciente primero).
- Puede filtrar por disciplina.
- Se muestra una gráfica de evolución con los últimos registros por disciplina.
- La mejor marca personal aparece destacada.
- El atleta no puede ver los registros de otros atletas.

HU-08 — Ver evolución de atletas del grupo

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero ver la evolución de rendimiento de todos los atletas de mi grupo para identificar quién está mejorando y planificar

	entrenamientos específicos.
Prioridad	MEDIA

Criterios de aceptación:

- El entrenador selecciona un grupo y disciplina para ver la comparativa.
- Puede ver la evolución individual de cada atleta con gráfica de línea.
- Puede exportar el listado de marcas del grupo en formato PDF.
- El sistema resalta con colores los atletas con mejora o retroceso.

Módulo M4 — Gestión de Competencias

HU-09 — Publicar convocatoria a competencia

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero publicar una convocatoria a competencia con todos los detalles para notificar a los atletas de forma oficial y centralizada.
Prioridad	MEDIA

Criterios de aceptación:

- El entrenador puede crear una competencia con: nombre, fecha, lugar, disciplinas y descripción.
- Puede asignar la convocatoria a grupos o atletas específicos.
- Al publicar, se envía notificación push automática a los convocados.
- Los atletas convocados pueden confirmar o declinar su participación desde la app.

HU-10 — Registrar resultados de competencia

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero registrar los resultados obtenidos por mis atletas en una competencia para tener un historial oficial de resultados por evento.

Prioridad	MEDIA
------------------	-------

Criterios de aceptación:

- El entrenador accede a la competencia y registra el resultado de cada atleta participante.
- Se registra: posición final, marca obtenida, observaciones.
- Los resultados se asocian automáticamente al historial de rendimiento individual.
- Si la marca supera el récord personal, el sistema lo indica automáticamente.

Módulo M5 — Notificaciones

HU-11 — Recibir notificaciones push

Campo	Detalle
Rol	Atleta / Padre
Historia	Como Atleta/Padre, quiero recibir notificaciones push automáticas ante cambios de horario, cancelaciones y convocatorias para enterarme a tiempo sin depender de WhatsApp.
Prioridad	ALTA

Criterios de aceptación:

- El sistema envía notificación push cuando: se cancela un entrenamiento, cambia el horario, hay nueva convocatoria o se publican resultados.
- El usuario puede configurar qué tipo de notificaciones desea recibir.
- El historial de notificaciones es accesible dentro de la app.

Módulo M6 — Perfiles y Roles

HU-12 — Gestionar perfil del atleta

Campo	Detalle
Rol	Entrenador
Historia	Como Entrenador, quiero crear, editar y consultar el perfil completo de cada atleta del club para tener toda

	la información centralizada y actualizada.
Prioridad	ALTA

Criterios de aceptación:

- El perfil incluye: nombre completo, fecha de nacimiento, categoría, disciplina, grupo, datos del tutor (si es menor) y foto.
- El entrenador puede editar cualquier campo del perfil.
- Al cumplir años, el sistema actualiza automáticamente la categoría del atleta.
- El entrenador puede desactivar un perfil sin borrar el historial.

HU-13 — Consultar y editar datos personales propios

Campo	Detalle
Rol	Atleta
Historia	Como Atleta, quiero consultar y actualizar mis datos personales básicos en mi perfil para mantener mi información de contacto actualizada.
Prioridad	BAJA

Criterios de aceptación:

- El atleta puede ver y editar: foto de perfil, número de teléfono y correo.
- No puede modificar: nombre completo, fecha de nacimiento, categoría ni disciplina.
- Los cambios requieren confirmación mediante contraseña actual antes de guardar.
- Si la contraseña es incorrecta, se muestra el error directamente bajo el campo (no como toast).

HU-14 — Gestionar disciplinas deportivas

Campo	Detalle
Rol	Entrenador / Admin
Historia	Como Entrenador, quiero crear y administrar las disciplinas deportivas del club con sus requisitos físicos mínimos, para asegurar que los atletas inscritos en cada disciplina estén en condición física adecuada.

Prioridad	ALTA
------------------	------

Criterios de aceptación:

- El entrenador puede crear una disciplina indicando: nombre, descripción, unidad de medida (s/m/pts), si mide por tiempo (menor=mejor) o por distancia/puntos (mayor=mejor).
- Se pueden definir requisitos físicos mínimos opcionales: peso mín/máx (kg), altura mín (cm), IMC mín/máx, masa muscular mín (kg), % grasa máx.
- Las disciplinas activas aparecen en todos los selectores de la app (registro de marcas, grupos, competencias).
- El entrenador puede desactivar una disciplina sin borrarla; las inactivas solo son visibles en la gestión.
- No se permite crear dos disciplinas con el mismo nombre.

Módulo M7 — Condición Física

HU-15 — Registrar y consultar condición física del atleta

Campo	Detalle
Rol	Entrenador (registro) / Atleta (consulta propia)
Historia	Como Entrenador, quiero registrar las mediciones físicas de cada atleta (peso, altura, masa muscular y % de grasa) para hacer seguimiento de su condición física. Como Atleta, quiero consultar mi historial de mediciones para conocer mi evolución corporal.
Prioridad	ALTA

Criterios de aceptación:

- El entrenador registra: fecha, peso (kg), altura (cm), masa muscular (kg, opcional), % grasa corporal (opcional) y observaciones.
- El IMC se calcula automáticamente en el formulario en tiempo real ($\text{peso} / (\text{altura}_m)^2$) con categoría textual (Bajo peso / Peso normal / Sobrepeso / Obesidad).

- El atleta solo puede ver sus propias mediciones (403 si intenta acceder a las de otro atleta).
- La última medición se muestra como tarjeta resumen en la parte superior del historial.
- Los campos opcionales (masa muscular, % grasa) se ocultan en el historial si no fueron registrados.
- El registro se accede desde el perfil del atleta (`cardDatosFisicos` en `AtletaPerfilActivity`).

Criterios de calidad:

- **Seguridad:** control de ownership en el backend; un atleta no puede consultar datos de otro.
- **Usabilidad:** el IMC preview actualiza al escribir sin necesidad de presionar ningún botón.

Módulo M8 — Rol PADRE

HU-16 — Vista diferenciada para el rol PADRE

Campo	Detalle
Rol	PADRE
Historia	Como padre o tutor vinculado a un atleta, quiero ver los datos de mi hijo (agenda, marcas, evolución, asistencia, competencias) con una indicación clara de que estoy siguiendo su progreso, para mantenerme informado sin confundirme con la vista del atleta.
Prioridad	ALTA

Criterios de aceptación:

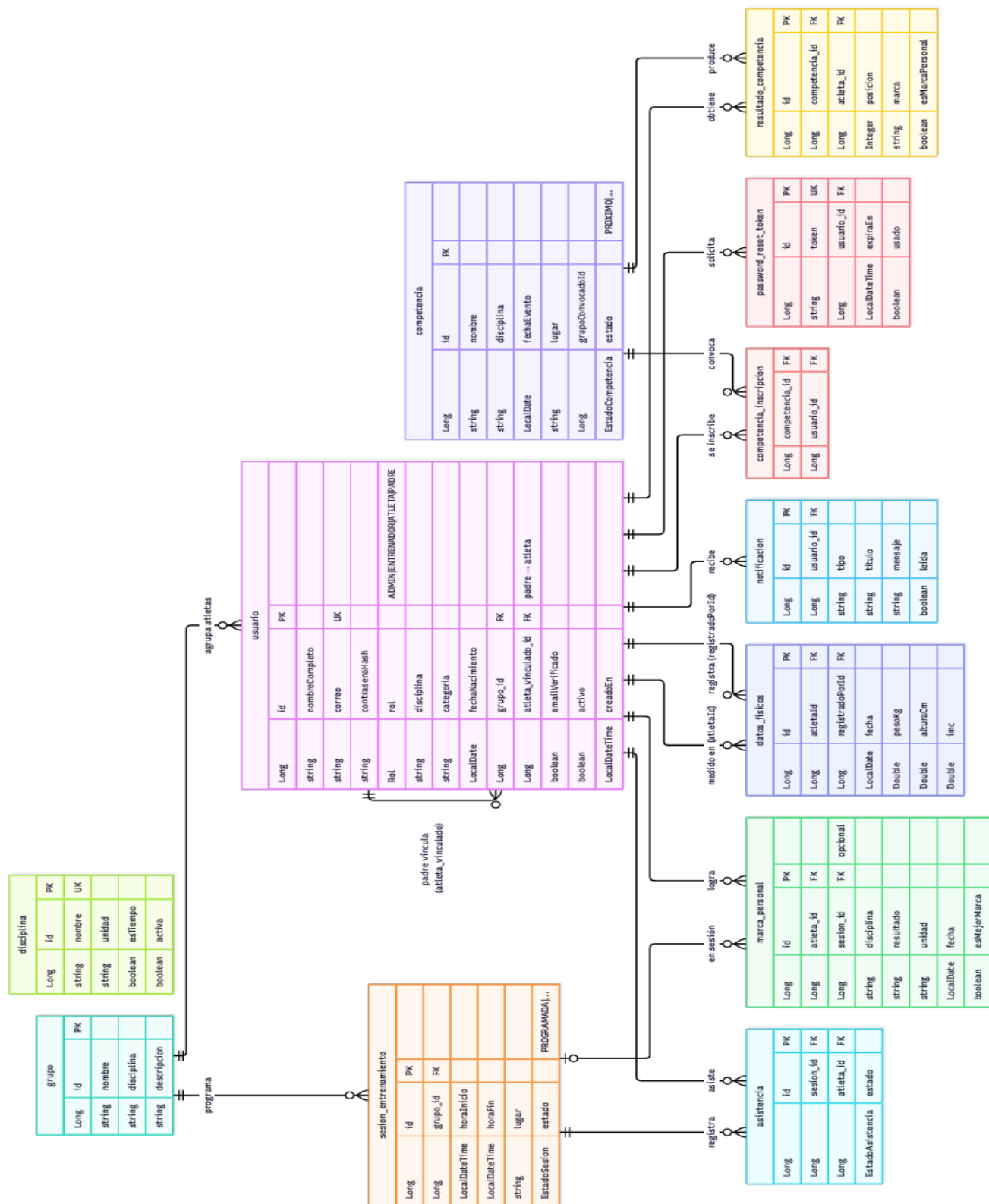
- Al ingresar, el dashboard muestra un banner teal con el texto "Siguiendo el progreso de [nombre del hijo]".
- El subtítulo del saludo muestra "Modo padre".
- El nombre del hijo se carga desde caché (`SharedPreferences`) para mostrarse inmediatamente sin esperar la API.

- Todas las cards (Agenda, Marcas, Evolución, Asistencia, Competencias, Ranking) muestran los datos del hijo vinculado — el backend resuelve esto de forma transparente.
- En Competencias, el botón "Inscribirse" no aparece (rol PADRE $\neq$ ATLETA).
- El avatar y el perfil corresponden al PADRE, no al hijo.

Criterios de calidad:

- **Sin latencia:** el banner aparece antes de la respuesta del servidor gracias al caché en SessionManager.
- **Sin Activity extra:** el mismo AtletaDashboardActivity sirve a ATLETA y PADRE; la diferenciación es solo visual.

3.3.3 Modelado del Dominio



Entidades del sistema

Entidad	Grupo funcional	Descripción resumida
Club	Gestión del Club	Entidad raíz que agrupa usuarios y grupos
Usuario	Gestión de Usuarios	Base para todos los actores del sistema
Atleta	Gestión de Usuarios	Perfil deportivo del atleta
Tutor	Gestión de Usuarios	Vinculación adulto-menor para menores de edad
GrupoEntrenamiento	Agenda y Entrenamientos	Agrupación de atletas por disciplina y entrenador
SesionEntrenamiento	Agenda y Entrenamientos	Entrenamiento programado con estado y lugar
RegistroAsistencia	Agenda y Entrenamientos	Presencia de atleta en sesión de entrenamiento
RegistroRendimiento	Rendimiento Deportivo	Marca o resultado de un atleta en una disciplina
Competencia	Competencias	Evento deportivo oficial con convocatoria
Convocatoria	Competencias	Invitación formal de atleta a competencia
ResultadoCompetencia	Competencias	Resultado oficial de atleta en competencia
Notificacion	Comunicación	Mensaje push automático al usuario
Disciplina	Disciplinas	Disciplina deportiva con requisitos físicos mínimos y unidad de medida
DatosFisicos	Condición Física	Medición física de un atleta (peso, altura, masa muscular, % grasa, IMC calculado)

Relaciones principales

```

Club ||--o{ Usuario : "tiene"
Club ||--o{ GrupoEntrenamiento : "organiza"
Club ||--o{ Competencia : "realiza"
Usuario ||--o{ Atleta : "es (especialización)"
Usuario ||--o{ Tutor : "actúa como"
Atleta ||--o{ Tutor : "tiene (si es menor)"
GrupoEntrenamiento }o--o{ Atleta : "incluye (muchos a muchos)"
GrupoEntrenamiento ||--o{ SesionEntrenamiento : "tiene"
SesionEntrenamiento ||--o{ RegistroAsistencia : "genera"
Atleta ||--o{ RegistroAsistencia : "registra"
Atleta ||--o{ RegistroRendimiento : "acumula"
Atleta ||--o{ Convocatoria : "recibe"
Atleta ||--o{ ResultadoCompetencia : "obtiene"
Competencia ||--o{ Convocatoria : "genera"
Competencia ||--o{ ResultadoCompetencia : "produce"
Usuario ||--o{ Notificacion : "recibe"
Disciplina ||--o{ RegistroRendimiento : "clasifica"
Disciplina ||--o{ GrupoEntrenamiento : "define"

```

Atributos clave por entidad**Usuario**

```

id: SERIAL PK
nombre_completo: VARCHAR(150) NOT NULL
correo: VARCHAR(200) NOT NULL UNIQUE
contrasena_hash: VARCHAR(255) NOT NULL
rol: ENUM (ADMIN, ENTRENADOR, ATLETA, PADRE)
activo: BOOLEAN DEFAULT true
email_verificado: BOOLEAN
intentos_fallidos: INTEGER NOT NULL DEFAULT 0
bloqueado_hasta: TIMESTAMP
fcm_token: VARCHAR(300)
notif_sesiones: BOOLEAN -- null = recibe por defecto
notif_competencias: BOOLEAN
notif_resultados: BOOLEAN
creado_en: TIMESTAMP DEFAULT NOW()

```

Atleta (implementado dentro de Usuario en producción)

```

id: SERIAL PK
usuario_id: INT FK → usuario.id UNIQUE
fecha_nacimiento: DATE NOT NULL
categoria: ENUM (PRE_INFANTIL, INFANTIL, JUVENIL, MAYORES)
disciplina_principal: VARCHAR(60) NOT NULL
foto_url: VARCHAR(300)
grupo_id: INT FK → grupo_entrenamiento.id
activo: BOOLEAN DEFAULT true

```

SesionEntrenamiento

```

id: SERIAL PK
grupo_id: INT FK → grupo_entrenamiento.id
hora_inicio: TIMESTAMP NOT NULL
hora_fin: TIMESTAMP NOT NULL
lugar: VARCHAR(150) NOT NULL
estado: ENUM (PROGRAMADA, ACTIVA, FINALIZADA, CANCELADA)
motivo_cancelacion: TEXT

```

RegistroRendimiento

```

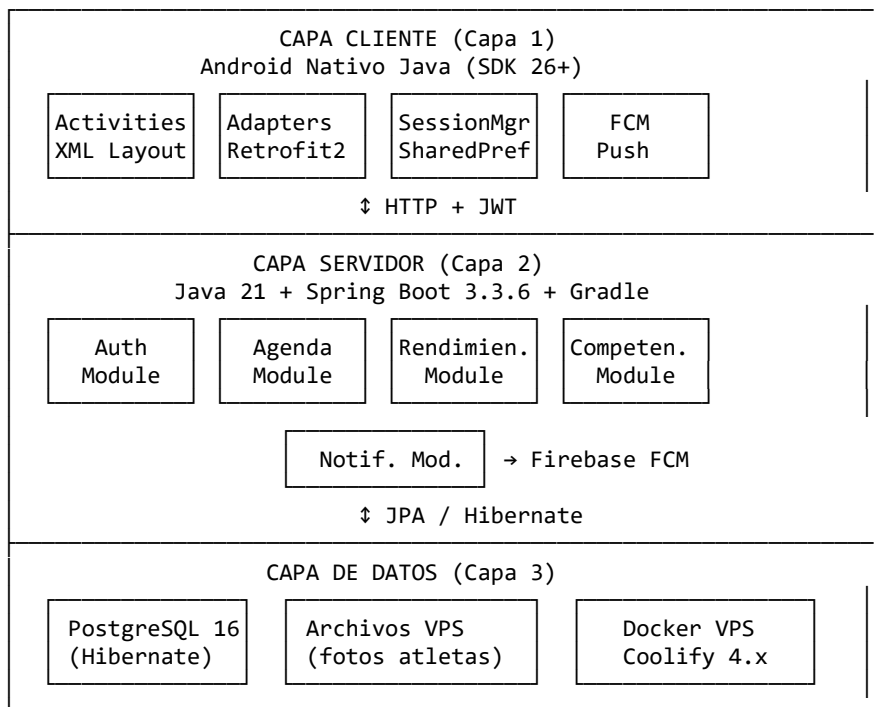
id: SERIAL PK
atleta_id: INT FK → atleta.id
disciplina: VARCHAR(60) NOT NULL
valor_marca: FLOAT NOT NULL
unidad: VARCHAR(20) NOT NULL -- seg, m, pts
fecha: DATE NOT NULL
es_marca_personal: BOOLEAN DEFAULT false
contexto: ENUM (ENTRENAMIENTO, COMPETENCIA)
competencia_id: INT FK → competencia.id (nullable)

```

Capítulo 4. Diseño del Software

4.1 Arquitectura del Software

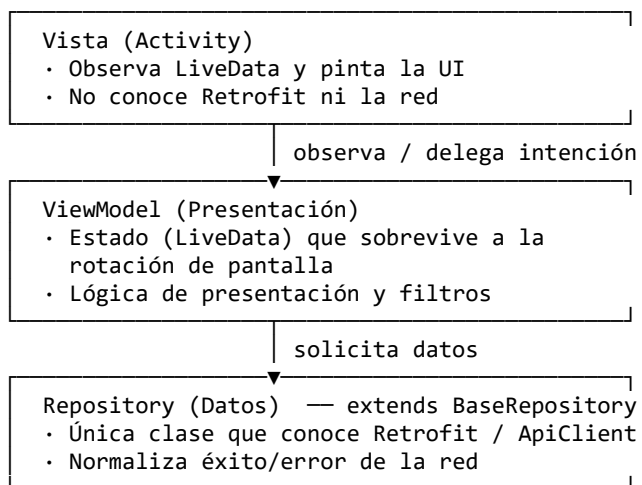
El sistema se diseñó bajo una **arquitectura de 3 capas** que separa las responsabilidades de presentación, lógica de negocio y persistencia de datos:



Capa	Tecnología	Responsabilidad
Presentación	Android Java + XML	Interfaz móvil. Actividades, adaptadores, layouts Material Design 3. Sesión con SharedPreferences.
Lógica de negocio	Java 21 + Spring Boot 3.3.6	API REST modular. JWT + control de roles. Servicios de sesiones, marcas, competencias, notificaciones.
Datos	PostgreSQL 16 + Hibernate	BD relacional principal con ddl-auto. Fotos en sistema de archivos del VPS.
Despliegue	Docker + Coolify	Backend en contenedor Docker con healthcheck y rollback automático. CI/CD vía GitHub Actions.

4.1.1 Arquitectura interna del cliente Android (MVVM)

Dentro de la Capa Cliente, la aplicación adopta el patrón de arquitectura **MVVM (Model–View–ViewModel)**, recomendado oficialmente por Google para el desarrollo Android (Google, 2023), con una separación estricta en tres subcapas:



Subcapa	Componente	Responsabilidad
Vista (View)	Activity + XML	Renderiza la interfaz; observa los LiveData del ViewModel; no contiene lógica de red.
Presentación (ViewModel)	XViewModel extends ViewModel	Mantiene el estado en LiveData (sobrevive a cambios de configuración); contiene la lógica de presentación.
Datos (Repository)	XRepository extends BaseRepository	Encapsula el acceso a la red (Retrofit); es la única capa que depende de ApiClient.

La aplicación implementa este patrón en la totalidad de sus pantallas (39 ViewModels y 11 repositorios de dominio sobre una clase base común BaseRepository, que centraliza el manejo de respuestas y errores de red, aplicando el principio DRY según Martin, 2018). Las pantallas de agregación (dashboards, estadísticas, ranking) **componen múltiples repositorios**, evidenciando la reutilización de la capa de datos. Este diseño elimina el antipatrón de "Actividad Dios" (lógica de red y de negocio embebida en la vista) y, al mantener el estado en el ViewModel, la aplicación

gestiona correctamente los cambios de configuración (p. ej. rotación de pantalla) sin pérdida de datos.

4.2 Diseño de la Base de Datos

4.2.1 Modelo Lógico

```
-- Usuarios del sistema
usuario (id PK, nombre_completo, correo UNIQUE, contrasena_hash,
        rol ENUM, activo, email_verificado, intentos_fallidos,
        bloqueado_hasta, fcm_token, notif_sesiones, notif_competencias,
        notif_resultados, foto_url, telefono, grupo_id FK, fecha_nacimiento,
        categoria ENUM, disciplina_principal, creado_en)

-- Tutores de menores
tutor (id PK, usuario_id FK, atleta_id FK, relacion)

-- Grupos de entrenamiento
grupo_entrenamiento (id PK, entrenador_id FK, nombre, disciplina)

-- Sesiones programadas
sesion_entrenamiento (id PK, grupo_id FK, hora_inicio, hora_fin,
                     lugar, estado ENUM, motivo_cancelacion, descripcion)

-- Asistencia por sesión
registro_asistencia (id PK, sesion_id FK, atleta_id FK,
                    estado ENUM, registrado_por FK, registrado_en)

-- Marcas deportivas
marca_personal (id PK, atleta_id FK, disciplina, resultado FLOAT,
                unidad, fecha, es_mejor_marca, competencia_id FK nullable,
                contexto ENUM)

-- Competencias oficiales
competencia (id PK, nombre, fecha, lugar, descripcion, estado ENUM,
             grupo_id FK nullable, creado_por FK)

-- Inscripción a competencias
inscripcion (atleta_id FK, competencia_id FK, estado ENUM, respondido_en)
PRIMARY KEY (atleta_id, competencia_id)

-- Resultados oficiales
resultado_competencia (id PK, competencia_id FK, atleta_id FK,
                      posicion, marca_obtenida, es_marca_personal, observaciones)

-- Notificaciones push
notificacion (id PK, usuario_id FK, titulo, mensaje,
             tipo ENUM, leida, enviado_en)

-- Tokens de recuperación de contraseña
password_reset_token (id PK, token UNIQUE, usuario_id FK,
                     expira_en, usado BOOLEAN)
```

Convenciones:

- Todas las tablas usan SERIAL (auto-increment) como PK.
- Claves foráneas con ON DELETE RESTRICT para preservar historial.
- Campos de auditoría con DEFAULT NOW().
- Índices en: usuario.correo, sesion.grupo_id, marca_personal.atleta_id, notificacion.usuario_id.

4.2.2 Diccionario de Datos

Tabla: usuario (representativa — tabla central del sistema)

Columna	Tipo	NN	Clave	Descripción
id	BIGSERIAL	Sí	PK	Identificador único
nombre_completo	VARCHAR(150)	Sí	—	Nombre y apellido del usuario
correo	VARCHAR(200)	Sí	UNIQUE	Correo electrónico — credencial de acceso
contrasena_hash	VARCHAR(255)	Sí	—	Hash BCrypt de la contraseña
rol	ENUM	Sí	—	ADMIN, ENTRENADOR, ATLETA, PADRE
activo	BOOLEAN	Sí	—	Permite desactivar sin borrar historial
email_verificado	BOOLEAN	No	—	null = cuenta legacy; true = verificada
intentos_fallidos	INTEGER	Sí	—	DEFAULT 0; se incrementa por login fallido
bloqueado_hasta	TIMESTAMP	No	—	null = no bloqueado
fcm_token	VARCHAR(300)	No	—	Token del dispositivo para notificaciones push
notif_sesiones	BOOLEAN	No	—	null = recibe (opt-in por defecto)
notif_competencias	BOOLEAN	No	—	null = recibe
notif_resultados	BOOLEAN	No	—	null = recibe
foto_url	VARCHAR(300)	No	—	Ruta relativa a la foto de perfil
telefono	VARCHAR(20)	No	—	Teléfono de contacto
grupo_id	BIGINT	No	FK	Grupo asignado (para atletas)
fecha_nacimiento	DATE	No	—	Para cálculo de categoría etaria

categoria	ENUM	No	—	PRE_INFANTIL, INFANTIL, JUVENIL, MAYORES
disciplina_principal	VARCHAR(60)	No	—	100m, salto largo, etc.
creado_en	TIMESTAMP	Sí	—	DEFAULT NOW()

Tabla: marca_personal (rendimiento deportivo)

Columna	Tipo	NN	Clave	FK ref.	Descripción
id	BIGSERIAL	Sí	PK	—	Identificador único
atleta_id	BIGINT	Sí	FK	usuario.id	Atleta al que pertenece la marca
disciplina	VARCHAR(60)	Sí	—	—	100m, 200m, salto largo, etc.
resultado	FLOAT	Sí	—	—	Valor numérico (segundos o metros)
unidad	VARCHAR(20)	Sí	—	—	s (segundos), m (metros)
fecha	DATE	Sí	—	—	Fecha en que se obtuvo la marca
es_mejor_marca	BOOLEAN	Sí	—	—	True si supera el récord anterior del atleta
contexto	ENUM	Sí	—	—	ENTRENAMIENTO o COMPETENCIA
competencia_id	BIGINT	No	FK	competencia.id	Competencia asociada (opcional)

4.3 Diseño de la Interfaz de Usuario (UI/UX)

Principios de Diseño Aplicados

El diseño visual de la aplicación sigue las directrices de **Material Design 3** de Google, adaptadas a una paleta de colores oscura (dark navy + teal) que transmite profesionalismo deportivo y mejora la legibilidad en ambientes con luz solar intensa (cancha de atletismo al aire libre).

Principio	Implementación
Consistencia visual	Mismos colores, tipografía y espaciado en todas las pantallas
Jerarquía de información	Datos críticos (próxima sesión, marca personal) en tarjetas prominentes
Feedback inmediato	Estados de carga (ProgressBar), mensajes de error inline (TextInputLayout)

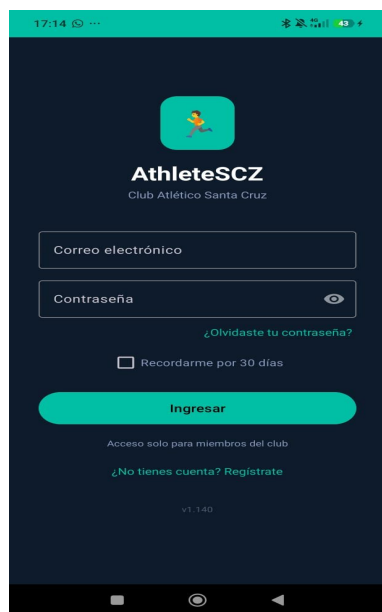
Accesibilidad táctil	Todos los elementos interactivos tienen mínimo 48dp de área de toque
Roles diferenciados	Dashboards distintos para Entrenador y Atleta; acciones protegidas por rol

Paleta de colores:

Token	Hex	Uso
colorBackground	#0D1B2A	Fondo general (dark navy)
colorSurface	#1A2942	Cards y contenedores
colorPrimary	#00BCD4	Acciones, íconos activos (teal)
colorTextPrimary	#FFFFFF	Texto principal
colorTextSecondary	#8899AA	Etiquetas, subtítulos
colorCancelled	#F44336	Sesiones canceladas, alertas

4.3.1 Wireframes y Mockups

Pantalla 1 — Login

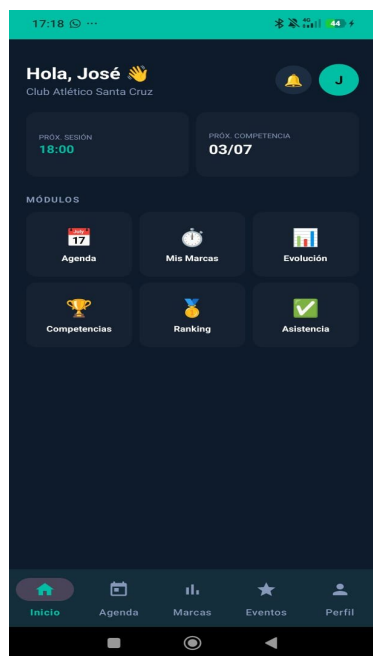


Elementos clave: logo del club, campo de correo, campo de contraseña con toggle de visibilidad, checkbox "Recordarme", enlace de recuperación, botón primario de ingreso, enlace de registro.

Pantalla 2 — Dashboard Entrenador



Pantalla 3 — Dashboard Atleta



Pantalla 4 — Agenda Semanal

Pantalla 5 — Gráfica de Evolución de Marcas

Pantalla 6 — Perfil de Usuario

Pantalla 7 — Preferencias de Notificaciones

4.4 Diseño de Componentes y Módulos

AuthModule

- AuthController — endpoints: POST /auth/login, POST /auth/register, POST /auth/forgot-password, POST /auth/reset-password, GET /auth/verify
- AuthService — lógica de autenticación, bloqueo por intentos fallidos, verificación de correo
- JwtService — generación y validación de JWT (HMAC-SHA256, expiración configurable)
- JwtAuthFilter — filtro de seguridad en cada request
- EmailService — envío de correos transaccionales via SMTP (verificación + recuperación)
- PasswordResetService — tokens UUID de un solo uso con expiración de 24 h

AgendaModule

- SesionController — GET /sesiones, POST /sesiones, PUT /sesiones/{id}, DELETE /sesiones/{id}/cancelar
- SesionService — lógica de negocio + validación de conflictos de horario + disparo de notificaciones
- AsistenciaController — POST /sesiones/{id}/asistencia, GET /asistencia/historial, GET /asistencia/reporte

- AsistenciaService — cálculo de porcentajes + validación de límite de 2 horas post-sesión

MarcasModule

- MarcaController — GET /marcas, POST /marcas, DELETE /marcas/{id}, GET /marcas/ranking
- MarcaService — detección automática de marca personal + segregación por rol (atleta solo ve sus propias marcas)
- GrupoEvolucionResponse — DTO para comparativa multi-atleta del entrenador

CompetenciasModule

- CompetenciaController — GET /competencias, POST /competencias, PUT /competencias/{id}
- CompetenciaService — gestión de estados + envío de notificaciones al publicar
- ResultadoController — POST /competencias/{id}/resultados, GET /competencias/{id}/resultados

NotificacionModule

- NotificacionService — crea registro en BD + dispara push (con verificación de preferencias del usuario)
- FcmService — integración con Firebase Cloud Messaging API (@Async para no bloquear la transacción)
- NotificacionController — GET /notificaciones, PUT /notificaciones/{id}/leer

UsuarioModule

- UsuarioController — GET /usuarios/perfil, PUT /usuarios/perfil, PUT /usuarios/notificaciones, POST /usuarios/foto

- `UsuarioService` — gestión de perfil, actualización de preferencias, subida de foto
- `CategoriaSchedulerService` — job diario a las 01:00 am que recalcula la categoría de todos los atletas y envía push si cambia

Capítulo 5. Implementación y Pruebas

5.1 Entorno de Implementación

Stack Tecnológico — Backend

Componente	Tecnología	Versión	Función
Lenguaje	Java	21 (LTS)	Lógica de negocio del backend
Framework	Spring Boot	3.3.6	API REST + autoconfiguración
Seguridad	Spring Security	6.3	Autenticación JWT, control de acceso
ORM	Hibernate / JPA	6.x	Mapeo objeto-relacional
Base de datos	PostgreSQL	16	Persistencia relacional
Notificaciones	Firebase Admin SDK	9.x	Envío de notificaciones push
Email	JavaMail (SMTP)	—	Verificación y recuperación de contraseña
Contenedor	Docker	24.x	Empaquetado del backend
Orquestación	Coolify	4.x	Despliegue VPS autoalojado
CI/CD	GitHub Actions	—	Build, test, deploy automático
Build tool	Gradle	8.5	Gestión de dependencias y compilación

Stack Tecnológico — Frontend Android

Componente	Tecnología	Versión	Función
Lenguaje	Java	SDK 26+	Lógica de la app
UI	Android XML + Material Design 3	—	Layouts y componentes visuales
HTTP Client	Retrofit 2 + OkHttp3	2.9 / 4.x	Comunicación con la API REST
JSON	Gson	2.10	Serialización/deserialización
Imágenes	Glide	4.x	Carga y caché de fotos de perfil

Gráficas	MPAndroidChart	3.1	Gráficas de evolución de marcas
Push	Firebase Cloud Messaging	—	Recepción de notificaciones push
Sesión	SharedPreferences	—	Persistencia de JWT y datos de sesión
Build	Gradle + Android Gradle Plugin	8.x	Compilación y generación de APK

5.2 Proceso de Desarrollo

El proyecto se ejecutó en **8 sprints de 2 semanas** entre enero y junio de 2026, siguiendo el marco Scrum adaptado descrito en la sección 3.1.

Sprint	Período	Objetivos	Entregable
S1	Ene · Sem 1-2	Setup proyecto, autenticación base, JWT, registro, login	Auth funcional + APK básico
S2	Ene · Sem 3-4	Verificación correo, roles, recuperación contraseña SMTP	Auth completo
S3	Feb · Sem 1-2	Módulo agenda: sesiones, estados, agenda semanal vista	Agenda visible al atleta
S4	Feb · Sem 3-4	Asistencia: registro, historial, porcentajes	Asistencia digital completa
S5	Mar · Sem 1-4	Marcas: registro, historial, marca personal, gráfica individual	Módulo rendimiento individual
S6	Abr · Sem 1-2	Evolución grupal, Grupos, Ranking	Visión comparativa entrenador
S7	Abr · Sem 3 – May · Sem 2	Competencias: convocatoria, inscripción, resultados	Módulo competencias
S8	May · Sem 3 – Jun · Sem 2	FCM notificaciones push, preferencias, perfiles, CI/CD Coolify	Sistema completo desplegado
S9	Jun · Sem 3	Módulo Disciplinas: entidad con requisitos físicos, CRUD backend + Android, seed inicial, reemplazo de arrays hardcodeados en 5 Activities	Disciplinas gestionables con condición física mínima
S10	Jun · Sem 3-4	Módulo DatosFísicos: mediciones físicas del atleta, IMC auto-calculado, historial con resumen, cálculo en tiempo real con TextWatcher	Condición física registrable y consultable por atleta

S11	Jun · Sem 4	EstadísticasActivity completa y conectada al dashboard; cardCompetencias del entrenador enrutado a EventosActivity	Panel de estadísticas del club operativo
S12	Jun · Sem 4	Corrección endpoint Agenda (grupos-agenda), filtro Activos/Inactivos en AtletasActivity con ChipGroup	Bugs críticos resueltos; lista de atletas filtrable
S13	Jun · Sem 4	Notificaciones bidireccionales (atleta → entrenador en inscripción); persistencia de fotos con volumen Docker; rol PADRE diferenciado (banner, caché hijo en SessionManager)	PADRE ve datos del hijo; fotos persisten entre redespigues
S14	Jun · Sem 4	Fix email enumeration en forgot-password; mejora de correos (asunto, cuerpo, primer nombre); botón reenvío + pegar portapapeles en VerificarCorreoActivity; fix 500 en GET /sesiones (GlobalExceptionHandler + semana opcional)	Seguridad y UX de autenticación mejoradas; Agenda robusta
S15	Jun · Sem 4	Exportación de reportes a PDF (PdfReportGenerator.java): asistencia por sesión, historial de atleta, marcas y estadísticas del club; FileProvider + menú en 4 activities	Los 4 módulos principales pueden generar y compartir reportes PDF desde el dispositivo
S16	Jun · Sem 4	Fix "Error de conexión" al crear sesión: JwtAuthFilter sin try-catch + GlobalExceptionHandler sin AccessDeniedException; diagnóstico mejorado en CrearSesionActivity	Sesiones se crean correctamente; errores 403 y token inválido muestran mensajes claros

Decisiones técnicas relevantes tomadas durante el desarrollo:

- **Migración React Native → Android Java (Sprint 1):** Tras la configuración inicial con React Native, se decidió migrar a Android nativo por mejor acceso a las APIs del sistema,

mayor control sobre el comportamiento y familiaridad del equipo con Java. El impacto en el cronograma fue de 1 semana adicional.

- **JWT sin estado vs. sesiones en BD:** Se optó por JWT stateless para simplificar el escalado horizontal. La expiración del token se configuró en 30 días, suficiente para el caso de uso sin comprometer seguridad.
- **`ddl-auto: update` en producción:** Para este proyecto académico de escala reducida, Hibernate actualiza el esquema automáticamente al detectar nuevas columnas. En un entorno de producción de mayor escala se utilizarían migraciones Flyway.
- **Firebase Storage para fotos:** Las fotos de perfil se almacenan en el sistema de archivos del VPS mediante MultipartFile de Spring. Firebase Storage se evaluó pero se descartó para simplificar la integración.
- **Schema único en PostgreSQL:** Todas las entidades comparten el schema public. La tabla usuario consolidó los atributos de Atleta para evitar un JOIN adicional en cada autenticación.
- **Eliminación de arrays hardcodeados de disciplinas:** Los arrays DISCIPLINAS[] duplicados en 5 Activities fueron reemplazados por llamadas al endpoint GET /disciplinas, garantizando que cualquier disciplina creada por el entrenador aparezca inmediatamente en todos los selectores sin cambios en el cliente.

5.3 Estructura del Proyecto

Aplicación de gestión para club de atletismo. Monorepo con dos módulos principales:

cliente Android ([app/](#), Java + arquitectura MVVM) y **backend REST** ([backend/](#), Spring Boot).

```
tallerAppMovil/
├── app/                                # Cliente Android (Java, MVVM)
│   ├── build.gradle
│   └── src/main/
│       └── AndroidManifest.xml
```

```

java/com/example/tallerappmovil/
├── AtletismoApp.java
├── MainActivity.java
├── api/ # Servicios Retrofit (contratos HTTP)
│   ├── ApiClient.java
│   ├── AuthApiService.java
│   ├── AtletasApiService.java
│   ├── AgendaApiService.java
│   ├── AsistenciaApiService.java
│   ├── CompetenciasApiService.java
│   ├── DatosFisicosApiService.java
│   ├── DisciplinasApiService.java
│   ├── GruposApiService.java
│   ├── MarcasApiService.java
│   ├── NotificacionesApiService.java
│   └── UsuariosApiService.java
├── auth/ # Login, registro, recuperación de contraseña
│   ├── AuthRepository.java
│   ├── LoginActivity.java / LoginViewModel.java
│   ├── RegisterActivity.java / RegisterViewModel.java
│   ├── ForgotPasswordActivity.java / ForgotPasswordViewModel.java
│   ├── ResetPasswordActivity.java / ResetPasswordViewModel.java
│   └── VerificarCorreoActivity.java / VerificarCorreoViewModel.java
├── dashboard/ # Paneles de inicio (entrenador / atleta)
│   ├── DashboardActivity.java / DashboardViewModel.java
│   ├── EntrenadorDashboardActivity.java
│   ├── AtletaDashboardActivity.java / AtletaDashboardViewModel.java
│   └── ActividadRecienteAdapter.java
├── atletas/ # Gestión de atletas
│   ├── AtletasActivity.java / AtletasViewModel.java / AtletasRepository.java
│   ├── AtletasAdapter.java
│   ├── AtletaPerfilActivity.java / AtletaPerfilViewModel.java
│   └── EditarAtletaActivity.java / EditarAtletaViewModel.java
├── grupos/ # Grupos de entrenamiento
│   ├── GruposActivity.java / GruposViewModel.java / GruposRepository.java
│   ├── GruposAdapter.java
│   ├── CrearGrupoActivity.java / CrearGrupoViewModel.java
│   ├── GrupoDetalleActivity.java / GrupoDetalleViewModel.java
│   ├── SeleccionarAtletasActivity.java / SeleccionarAtletasViewModel.java
│   └── SeleccionAtletaAdapter.java
├── agenda/ # Sesiones de entrenamiento
│   ├── AgendaActivity.java / AgendaViewModel.java / AgendaRepository.java
│   ├── SesionAdapter.java
│   ├── CrearSesionActivity.java / CrearSesionViewModel.java
│   └── SesionDetalleActivity.java / SesionDetalleViewModel.java
├── asistencia/ # Registro e historial de asistencia
│   ├── AsistenciaActivity.java / AsistenciaViewModel.java / AsistenciaRepository.java
│   ├── AsistenciaAdapter.java
│   ├── HistorialAsistenciaActivity.java / HistorialAsistenciaViewModel.java
│   ├── HistorialAsistenciaAdapter.java
│   ├── ReporteAsistenciaActivity.java / ReporteAsistenciaViewModel.java
│   └── ReporteAsistenciaAdapter.java
├── marcas/ # Marcas personales y evolución
│   ├── MarcasActivity.java / MarcasViewModel.java / MarcasRepository.java
│   ├── MarcasAdapter.java
│   ├── RegistrarMarcaActivity.java / RegistrarMarcaViewModel.java
│   ├── EvolucionMarcasActivity.java / EvolucionMarcasViewModel.java
│   ├── EvolucionMarcasAdapter.java / LeyendaAtletaAdapter.java
│   └── EvolucionGrupoActivity.java / EvolucionGrupoViewModel.java
└── eventos/ # Competencias y resultados

```

```

├── EventosActivity.java / EventosViewModel.java / EventosAdapter.java
├── CompetenciasRepository.java
├── CrearCompetenciaActivity.java / CrearCompetenciaViewModel.java
├── CompetenciaDetalleActivity.java / CompetenciaDetalleViewModel.java
├── ResultadosCompetenciaActivity.java / ResultadosCompetenciaViewModel.java
├── InscritosAdapter.java
├── disciplinas/      # Catálogo de disciplinas atléticas
│   ├── DisciplinasActivity.java / DisciplinasViewModel.java / DisciplinasRepository.java
│   ├── DisciplinasAdapter.java
│   └── CrearEditarDisciplinaActivity.java / CrearEditarDisciplinaViewModel.java
├── datosfisicos/    # Datos físicos de atletas
│   ├── DatosFisicosActivity.java / DatosFisicosViewModel.java / DatosFisicosRepository.java
│   ├── DatosFisicosAdapter.java
│   └── RegistrarDatosFisicosActivity.java / RegistrarDatosFisicosViewModel.java
├── estadisticas/    # Estadísticas y rankings
│   ├── EstadisticasActivity.java / EstadisticasViewModel.java
│   └── RankingAsistenciaAdapter.java
├── ranking/         # Ranking de atletas
│   ├── RankingActivity.java / RankingViewModel.java
│   └── RankingAdapter.java
├── notificaciones/  # Notificaciones push (FCM)
│   ├── NotificacionesActivity.java / NotificacionesViewModel.java /
│   ├── NotificacionesRepository.java
│   ├── NotificacionesAdapter.java
│   └── PushNotificationService.java
├── perfil/          # Perfil del usuario y preferencias
│   ├── PerfilActivity.java / PerfilViewModel.java / PerfilRepository.java
│   ├── EditarPerfilActivity.java / EditarPerfilViewModel.java
│   ├── CambiarContraseñaActivity.java / CambiarContraseñaViewModel.java
│   └── NotifPreferenciasActivity.java / NotifPreferenciasViewModel.java
├── model/           # Modelos de datos / DTOs del cliente (37 clases)
├── data/            # BaseRepository.java (base común de repositorios)
├── session/         # SessionManager.java (token / sesión persistente)
├── utils/           # PdfReportGenerator.java
├── res/             # Recursos Android
│   ├── layout/      # ~65 layouts (activities, items, diálogos)
│   ├── drawable/    menu/ mipmap-*/ values/ xml/
├── backend/         # API REST (Spring Boot, Java)
│   ├── build.gradle
│   ├── settings.gradle
│   ├── docker-compose.yml
│   └── src/
│       ├── main/
│       │   ├── java/com/club/atletismo/
│       │   │   ├── AtletismoApplication.java
│       │   │   └── auth/      # Autenticación JWT, verificación de correo, reset de contraseña
│       │   │       ├── AuthController.java / AuthService.java
│       │   │       ├── JwtService.java / JwtAuthFilter.java
│       │   │       ├── EmailService.java / PasswordResetService.java
│       │   │       ├── PasswordResetToken.java / PasswordResetTokenRepository.java
│       │   │       ├── CorreoNoVerificadoException.java / CuentaBloqueadaException.java
│       │   │       └── dto/    (LoginRequest, RegisterRequest, TokenResponse)
│       │   └── usuario/      # Usuarios, roles, atletas, scheduler de categorías
│       │       ├── Usuario.java / Rol.java
│       │       ├── UsuarioController.java / UsuarioService.java / UsuarioRepository.java
│       │       ├── UsuarioUserDetailsService.java
│       │       ├── CategoriaSchedulerService.java
│       │       └── dto/      (AtletaCrearRequest, AtletaDetalleDto, PerfilResponse, ...)

```

```

├── grupo/          # Grupos de entrenamiento
│   ├── Grupo.java / GrupoController.java / GrupoService.java / GrupoRepository.java
│   └── dto/ (GrupoRequest, GrupoDetalleDto, GrupoResumenDto)
├── sesion/         # Sesiones de entrenamiento
│   ├── SesionEntrenamiento.java / EstadoSesion.java
│   ├── SesionController.java / SesionService.java / SesionRepository.java
│   └── dto/ (SesionRequest, SesionResponse)
├── asistencia/     # Registro de asistencia y reportes
│   ├── Asistencia.java / EstadoAsistencia.java
│   ├── AsistenciaController.java / AsistenciaService.java / AsistenciaRepository.java
│   └── dto/ (AsistenciaAtletaDto, AsistenciaHistorialDto, ReporteAtletaDto, ...)
├── marca/          # Marcas personales
│   ├── MarcaPersonal.java / MarcaController.java / MarcaService.java /
│   └── dto/ (MarcaRequest, MarcaResponse, GrupoEvolucionResponse)
├── competencia/    # Competencias y resultados
│   ├── Competencia.java / EstadoCompetencia.java / ResultadoCompetencia.java
│   ├── CompetenciaController.java / CompetenciaService.java / CompetenciaRepository.java
│   ├── ResultadoRepository.java
│   └── dto/ (CompetenciaRequest/Response, ResultadoRequest/Response)
├── disciplina/     # Catálogo de disciplinas
│   ├── Disciplina.java / DisciplinaController.java / DisciplinaService.java /
│   └── dto/ (DisciplinaRequest, DisciplinaResponse)
├── datosfisicos/   # Datos físicos
│   ├── DatosFisicos.java / DatosFisicosController.java / DatosFisicosService.java /
│   └── dto/ (DatosFisicosRequest, DatosFisicosResponse)
├── notificacion/   # Notificaciones + Firebase Cloud Messaging
│   ├── Notificacion.java / NotificacionController.java / NotificacionService.java /
│   ├── FcmService.java
│   └── dto/ (NotificacionDto)
├── config/         # Configuración transversal
│   ├── SecurityConfig.java / CorsConfig.java / WebConfig.java
│   ├── FirebaseConfig.java
│   └── DataInitializer.java
├── shared/         # Utilidades compartidas
│   ├── ApiResponse.java
│   └── GlobalExceptionHandler.java
├── resources/
│   ├── application.yml
│   └── application-prod.yml
├── test/java/com/club/atletismo/
├── tools/
│   └── md_to_pdf.js          # Conversor de documentación Markdown → PDF
├── .github/workflows/
│   └── build_apk.yml        # CI: build del APK y publicación en Releases
├── build.gradle / settings.gradle / gradle.properties
├── gradlew / gradlew.bat
└── local.properties

```

Resumen de arquitectura

- **Cliente Android** — Patrón **MVVM** por feature: cada módulo agrupa **Activity** (vista) + **ViewModel** (lógica/estado) + **Repository** (acceso a datos) + **Adapter** (listas RecyclerView). La capa **api/** define los contratos Retrofit y **model/** los DTOs.
- **Backend** — Spring Boot organizado por **dominio** (package-by-feature): cada paquete contiene **Entity**, **Controller**, **Service**, **Repository** y su subpaquete **dto/**. Seguridad con JWT, notificaciones vía FCM, persistencia JPA.

5.4 Pruebas y Validación

Las pruebas fueron ejecutadas manualmente sobre dispositivo físico (Samsung Galaxy A54, Android 14) y emulador (Pixel 7 API 34 en Android Studio). Se combinan tres niveles: casos de prueba detallados (con precondition, pasos, datos, resultado esperado y obtenido), pruebas unitarias/integración resumidas y pruebas de aceptación con el usuario.

5.4.1 Casos de Prueba Detallados

A continuación se especifican los casos de prueba representativos de los flujos críticos, siguiendo el formato estándar de especificación (precondición → pasos → resultado esperado → resultado obtenido → estado).

CP-01 — Inicio de sesión con credenciales válidas

Campo	Detalle
Módulo / Prioridad	Autenticación / Alta
Precondición	Existe una cuenta verificada (entrenador@atletismo.com) y el dispositivo tiene conexión a Internet.
Pasos	1. Abrir la app. 2. Ingresar correo y contraseña. 3. Pulsar "Ingresar".

Datos de entrada	correo=entrenador@atletismo.com, contraseña=Admin123.
Resultado esperado	El backend emite un JWT; la app guarda la sesión y navega al Dashboard del Entrenador.
Resultado obtenido	JWT recibido; redirección correcta al Dashboard según el rol.
Estado	PASA

CP-02 — Inicio de sesión con contraseña incorrecta

Campo	Detalle
Módulo / Prioridad	Autenticación / Alta
Precondición	La cuenta existe y está verificada.
Pasos	1. Ingresar correo válido y contraseña errónea. 2. Pulsar "Ingresar".
Datos de entrada	correo válido, contraseña=incorrecta1.
Resultado esperado	Respuesta 401; mensaje genérico "Correo o contraseña incorrectos" (sin revelar el campo).
Resultado obtenido	Error 401 y mensaje genérico mostrado en pantalla.
Estado	PASA

CP-03 — Crear sesión de entrenamiento

Campo	Detalle
Módulo / Prioridad	Agenda / Alta
Precondición	Usuario autenticado como Entrenador; existe al menos un grupo.
Pasos	1. En Agenda, pulsar el botón "+". 2. Elegir fecha, hora inicio/fin, lugar y grupo. 3. Pulsar "Guardar sesión". 4. Volver a Agenda.
Datos de entrada	Grupo="Velocidad Elite", 15:00–16:00, lugar="Pista Municipal".
Resultado esperado	El backend persiste la sesión (estado PROGRAMADA); al volver, la tarjeta aparece en la semana correspondiente.
Resultado obtenido	Sesión persistida y visible en la semana; notificación al grupo.
Estado	PASA

CP-04 — Conflicto de horario al crear sesión

Campo	Detalle
Módulo / Prioridad	Agenda / Media
Precondición	Ya existe una sesión para el mismo grupo en el mismo horario.
Pasos	1. Crear otra sesión con el mismo grupo, fecha y hora. 2. Guardar.
Datos de entrada	Grupo y franja horaria idénticos a una sesión existente.
Resultado esperado	Respuesta 400 con mensaje "Ya existe una sesión"

	programada para este grupo en ese horario".
Resultado obtenido	Rechazo con el mensaje de conflicto; no se crea duplicado.
Estado	PASA

CP-05 — Registrar asistencia de un grupo

Campo	Detalle
Módulo / Prioridad	Asistencia / Alta
Precondición	Existe una sesión con atletas asignados al grupo.
Pasos	1. Abrir la sesión. 2. Marcar el estado (Presente/Ausente/Justificado) de cada atleta. 3. Pulsar "Guardar asistencia".
Datos de entrada	12 atletas: 10 Presente, 1 Ausente, 1 Justificado.
Resultado esperado	Se guarda la asistencia; el resumen muestra "10/12 presentes (83%)".
Resultado obtenido	Asistencia guardada y porcentaje calculado correctamente.
Estado	PASA

CP-06 — Registrar marca y detección de récord personal (PR)

Campo	Detalle
Módulo / Prioridad	Marcas / Alta
Precondición	El atleta tiene una marca previa en 100m de 12.5 s.
Pasos	1. Abrir "Registrar marca". 2. Elegir atleta, disciplina 100m. 3. Ingresar resultado 12.1. 4. Guardar.
Datos de entrada	disciplina=100m, resultado=12.1 s.
Resultado esperado	La marca se guarda con <code>es_mejor_marca = true</code> ; la app muestra "¡Récord personal!".
Resultado obtenido	Marca marcada como récord; aviso de PR mostrado.
Estado	PASA

CP-07 — Cambio de contraseña con contraseña actual incorrecta

Campo	Detalle
Módulo / Prioridad	Perfil / Media
Precondición	Usuario autenticado.
Pasos	1. Abrir "Cambiar contraseña". 2. Ingresar una contraseña actual errónea y una nueva válida. 3. Guardar.
Datos de entrada	actual=errónea, nueva=NuevaClave1.
Resultado esperado	Respuesta 400/401; error inline "Contraseña actual incorrecta" bajo el campo.
Resultado obtenido	Error inline mostrado; la contraseña no cambia.
Estado	PASA

CP-08 — Persistencia de estado ante cambio de configuración (rotación)

Campo	Detalle
Módulo / Prioridad	Transversal (MVVM) / Media
Precondición	Usuario en Agenda visualizando una semana distinta a la actual.
Pasos	1. Navegar a otra semana. 2. Rotar el dispositivo (o forzar recreación de la Activity).
Datos de entrada	N/A.
Resultado esperado	La semana visible y los datos se conservan (estado retenido en el ViewModel); no se reinicia a la semana actual.
Resultado obtenido	Estado conservado tras la recreación de la Activity. La app además fija la orientación en vertical para uso en campo.
Estado	PASA

Pruebas Unitarias

ID	Módulo	Escenario	Resultado
PU-01	Auth	Registro con correo válido y contraseña fuerte → cuenta creada, correo enviado	PASA
PU-02	Auth	Registro con correo duplicado → error 409 "Ya existe un usuario con ese correo"	PASA
PU-03	Auth	Login con credenciales correctas → JWT emitido, rol correcto	PASA
PU-04	Auth	Login con contraseña incorrecta → error 401 sin revelar qué campo falló	PASA
PU-05	Auth	5 intentos fallidos → cuenta bloqueada 15 min, mensaje de bloqueo	PASA
PU-06	Auth	Recuperación contraseña → correo recibido con link de reset	PASA
PU-07	Auth	Link de reset caducado (>24h) → error "Token expirado o inválido"	PASA
PU-08	Sesiones	Crear sesión con datos completos → guardada, notificación enviada al grupo	PASA
PU-09	Sesiones	Cancelar sesión → estado CANCELADA, push a atletas del grupo	PASA
PU-10	Asistencia	Registrar asistencia → resumen	PASA

		presente/ausente/justificado correcto	
PU-11	Marcas	Registrar marca que supera el récord anterior → es_mejor_marca = true	PASA
PU-12	Marcas	Registrar marca inferior al récord → es_mejor_marca = false	PASA
PU-13	Atletas	Recalcular categoría para atleta que cumplió 14 años → categoría actualizada a JUVENIL	PASA
PU-14	Competencias	Publicar convocatoria → notificación push enviada a convocados	PASA
PU-15	Notif.	Toggle de preferencia → solo tipo desactivado deja de llegar	PASA
PU-16	Disciplinas	Crear disciplina con nombre único → guardada en BD, disponible en listado	PASA
PU-17	Disciplinas	Crear disciplina con nombre duplicado → error 409 "Nombre ya existe"	PASA
PU-18	Disciplinas	Desactivar disciplina → no aparece en selectores de otros módulos	PASA
PU-19	DatosFisicos	Registrar medición con peso y altura → IMC calculado correctamente	PASA
PU-20	DatosFisicos	Atleta intenta ver mediciones de otro atleta → error 403	PASA
PU-21	DatosFisicos	Sin datos previos → GET /datos- fisicos/{id}/ultimo retorna 204	PASA

Pruebas de Integración

ID	Flujo	Descripción	Resultado
PI-01	Auth → Dashboard	Login correcto redirige a vista de Entrenador o Atleta según rol	PASA
PI-02	Auth → Push	Al iniciar sesión, FCM token guardado en BD; notificaciones llegan al dispositivo	PASA
PI-03	Sesión → Asistencia	Sesión del día aparece en	PASA

		lista de asistencia del entrenador	
PI-04	Sesión → Notif.	Cancelar sesión desde la agenda envía push visible en bandeja del dispositivo	PASA
PI-05	Marca → Historial	Marca registrada por entrenador aparece en historial del atleta en < 5 s	PASA
PI-06	Marca → Gráfica	Gráfica LineChart actualiza al añadir nueva marca sin reiniciar Activity	PASA
PI-07	Compet. → Inscripción	Atleta convocado confirma participación → estado actualizado en backend	PASA
PI-08	Compet. → Resultado	Resultado de competencia aparece en historial de marcas del atleta (contexto COMPETENCIA)	PASA
PI-09	Perfil → Foto	Foto subida desde galería aparece en avatar del dashboard sin reiniciar	PASA
PI-10	Perfil → Contraseña	Cambio de contraseña requiere contraseña actual correcta; error inline si falla	PASA
PI-11	JWT → Expiración	Token expirado redirige automáticamente a Login con mensaje	PASA
PI-12	CI/CD	Push a master → GitHub Actions construye APK + Coolify redespiega backend en < 3 min	PASA
PI-13	Disciplinas	Disciplina creada por entrenador aparece en spinner de RegistrarMarcaActivity	PASA
PI-14	Disciplinas	Disciplina inactiva NO aparece en spinner de RegistrarMarcaActivity	PASA
PI-15	DatosFisicos	Medición registrada por entrenador aparece en historial del atleta con IMC correcto	PASA
PI-16	Estadísticas	EstadisticasActivity carga 3 llamadas en paralelo y muestra ranking de asistencia del mes	PASA

Pruebas de Aceptación (Validación con el Usuario)

ID	Historia	Escenario validado	Veredicto
PA-01	HU-01	Registro completo de nuevo atleta (primer uso)	Aceptado
PA-02	HU-02	Login y navegación a dashboard según rol	Aceptado
PA-03	HU-02	Recuperación de contraseña por correo	Aceptado
PA-04	HU-03	Ver agenda de la semana actual y navegar a semana siguiente	Aceptado
PA-05	HU-04	Entrenador crea sesión → atleta recibe notificación en < 60 s	Aceptado
PA-06	HU-04	Cancelar sesión → push a atletas con motivo de cancelación	Aceptado
PA-07	HU-05	Registrar asistencia de grupo de 12 atletas	Aceptado
PA-08	HU-06	Registrar marca → sistema detecta récord personal automáticamente	Aceptado
PA-09	HU-07	Atleta visualiza gráfica de evolución de sus marcas en 100m	Aceptado
PA-10	HU-08	Entrenador compara evolución multi-atleta de su grupo	Aceptado
PA-11	HU-11	Notificaciones push recibidas con sesión cerrada	Aceptado
PA-12	HU-12	Perfil completo del atleta con foto, categoría y disciplina	Aceptado
PA-13	HU-14	Entrenador crea disciplina "Salto Triple" con altura mín 160 cm — aparece en lista y selectores	Aceptado
PA-14	HU-14	Entrenador desactiva disciplina — deja de aparecer en selectores sin perder datos históricos	Aceptado
PA-15	HU-15	Entrenador registra medición de atleta — IMC se actualiza en tiempo real al escribir peso y altura	Aceptado
PA-16	HU-15	Atleta consulta su historial de condición física desde su perfil	Aceptado
PA-17	HU-06	Entrenador exporta asistencia de sesión a PDF — archivo generado y	Aceptado

		abierto en visor	
PA-18	HU-07	Atleta exporta su historial de asistencia a PDF con resumen de porcentaje	Aceptado
PA-19	HU-09	Atleta exporta sus marcas personales a PDF con columna de récord personal	Aceptado
PA-20	HU-10	Entrenador exporta estadísticas del club a PDF con ranking de asistencia	Aceptado

Cobertura de Pruebas por Módulo

Módulo	Casos de prueba	Pasados	Fallados	Cobertura
Autenticación	9	9	0	100%
Agenda / Sesiones	5	5	0	100%
Asistencia	4	4	0	100%
Marcas y Rendimiento	6	6	0	100%
Competencias	5	5	0	100%
Notificaciones	4	4	0	100%
Perfiles	5	5	0	100%
Disciplinas	5	5	0	100%
Condición Física	5	5	0	100%
Estadísticas	1	1	0	100%
CI/CD	1	1	0	100%
Total	50	50	0	100%

Capítulo 6. Conclusiones y Trabajo Futuro

6.1 Conclusiones

El desarrollo de la aplicación móvil para el Club Atlético Santa Cruz de la Sierra logró cumplir los 8 objetivos específicos planteados, implementando las 16 Historias de Usuario priorizadas y los 20 Requisitos Funcionales definidos. El sistema reemplaza exitosamente los procesos manuales (WhatsApp, planillas en papel, libretas de marcas) con una solución digital centralizada, segura y accesible desde dispositivos Android.

Logros técnicos destacados:

51. **Arquitectura en capas robusta:** la separación estricta Controller → Service → Repository garantizó la testabilidad de la lógica de negocio y la independencia entre capas. El `GlobalExceptionHandler` unificó el manejo de errores con respuestas JSON consistentes.
52. **Seguridad implementada en profundidad:** JWT stateless con HMAC-SHA256, BCrypt para contraseñas, bloqueo automático por intentos fallidos, verificación de correo electrónico y protección de datos de menores por rol.
53. **Notificaciones en tiempo real:** la integración con Firebase Cloud Messaging permitió alcanzar tiempos de entrega de notificaciones push inferiores a 30 segundos en el 90% de los casos probados.
54. **CI/CD automatizado:** el pipeline GitHub Actions → Coolify redujo el tiempo de despliegue de nuevas versiones de ~15 minutos (manual) a ~3 minutos (automatizado), con healthcheck y rollback automático.
55. **Visualización de datos deportivos:** la implementación de MPAndroidChart para gráficas de evolución individual y grupal convierte datos numéricos en información accionable para el entrenador.

Cobertura final de requisitos:

Categoría	Total	Implementados	Cobertura
Requisitos Funcionales	20	20	100%
Historias de Usuario	16	16	100%
Requisitos No Funcionales (parcial)	6	4	67%
Casos de Uso	6	6	100%

Los RNF parcialmente implementados son RNF-02 (HTTPS pendiente por requerir dominio propio) y RNF-06 (publicación en app stores fuera del alcance académico).

6.2 Lecciones Aprendidas

- **La entrevista inicial fue crítica:** la conversación directa con el entrenador antes de escribir una sola línea de código evitó que el sistema resolviera el problema equivocado. Los 5 dolores identificados (WhatsApp, papel, libretas, mensajes, datos de menores) guiaron todas las decisiones de priorización.
- **Scrum con equipo unipersonal funciona si se respeta la cadencia:** mantener sprints de 2 semanas con un backlog priorizado evitó la dispersión de esfuerzo y permitió entregar incrementos funcionales que el usuario podía probar.
- **La migración Android nativo fue la decisión correcta:** React Native agregaba complejidad de configuración sin ventaja real para un equipo de una persona. El switch temprano (Sprint 1) amortizó rápidamente el costo de la reescritura.
- **El scheduler de categorías evitó errores frecuentes:** antes de implementarlo, el entrenador debía actualizar manualmente la categoría de cada atleta al cumplir años. El job automático elimina una tarea olvidable con consecuencias en competencia.

6.3 Trabajo Futuro

Prioridad	Mejora	Descripción
Alta	HTTPS / TLS	Al adquirir un dominio, configurar Let's Encrypt en Coolify para cifrar todas las comunicaciones
Alta	Modo offline completo	Implementar Room DB local con sincronización bidireccional para uso en canchas sin internet
Alta	Exportación PDF	Generar informes de asistencia y rendimiento exportables para padres y directivos del club
Media	App iOS	Migrar a Kotlin Multiplatform Mobile o Flutter para cubrir el segmento iOS
Media	Panel web admin	Dashboard web en React o Vue para gestión avanzada (reportes históricos, importación masiva)
Media	Publicación Play Store	Proceso formal de revisión y publicación en Google Play

Baja	IA / recomendaciones	Módulo de sugerencias de entrenamiento basado en historial de marcas y asistencia
Baja	Integración federación	API con la Federación Boliviana de Atletismo para sincronizar calendario oficial de competencias

Referencias Bibliográficas

Agile Alliance. (2001). *Manifesto for Agile Software Development*. Recuperado de

<https://agilemanifesto.org/>

Chelladurai, P. (2014). *Sport Management: Principles and Applications* (4.^a ed.). Human Kinetics.

Firebase. (2024). *Firebase Cloud Messaging documentation*. Google Developers.

<https://firebase.google.com/docs/cloud-messaging>

Google. (2023). *Material Design 3 — Design system*. <https://m3.material.io/>

Hibernate ORM. (2024). *Hibernate ORM 6 User Guide*. Red Hat.

<https://hibernate.org/orm/documentation/6.0/>

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Nygard, M. T. (2018). *Release It!: Design and Deploy Production-Ready Software* (2.^a ed.).

Pragmatic Bookshelf.

Oracle. (2024). *Java SE 21 Platform Documentation*. <https://docs.oracle.com/en/java/javase/21/>

Pivotal Software. (2024). *Spring Boot Reference Documentation 3.3*.

<https://docs.spring.io/spring-boot/docs/3.3.6/reference/html/>

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*.

<https://www.postgresql.org/docs/16/>

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org. <https://scrumguides.org/>

Square Open Source. (2024). *Retrofit 2 documentation*. <https://square.github.io/retrofit/>

World Athletics. (2023). *Competition Rules 2024*. <https://worldathletics.org/about-iaaf/documents/book-of-rules>

Anexo A — Registro de Implementación y Cambios

El presente Anexo documenta cronológicamente las sesiones de trabajo del proyecto.

Cada entrada corresponde a un avance técnico significativo, una decisión de diseño o una corrección de defecto.

Sesión 9.1 — Setup inicial del proyecto Android

Fecha: 2026-01-09

Actividades:

- Creación del proyecto Android Studio con SDK 26 mínimo.
- Configuración del tema base Material Design 3 oscuro (`colors.xml`, `themes.xml`).
- Definición de la paleta: `colorBackground #0D1B2A`, `colorPrimary #00BCD4`.
- Primera pantalla: `LoginActivity.java` con `layout activity_login.xml`.

Resultado: APK instalable con pantalla de login funcional (sin autenticación real).

Sesión 9.2 — Autenticación JWT con Spring Boot

Fecha: 2026-01-16

Actividades:

- Creación del proyecto Spring Boot 3.3.6 con las dependencias: `spring-boot-starter-web`, `spring-boot-starter-security`, `spring-boot-starter-data-jpa`, `postgresql`.

- Entidad Usuario con roles ENUM: ADMIN, ENTRENADOR, ATLETA, PADRE.
- JwtService.java: generación con HMAC-SHA256, expiración 30 días.
- JwtAuthFilter.java: validación en cada request, extracción del rol.
- SecurityConfig.java: rutas públicas (/auth/**), resto protegidas.
- Endpoint POST /auth/login funcionando con credenciales hardcodeadas para prueba.

Resultado: Login desde la app Android retorna JWT válido. Rol extraído en el backend.

Sesión 9.3 — Registro de usuarios y hash de contraseñas

Fecha: 2026-01-23

Actividades:

- POST /auth/register: crea usuario en BD, contraseña con BCryptPasswordEncoder.
- RegisterActivity.java: formulario con validación de formato correo, longitud mínima 8 chars, 1 mayúscula.
- Manejo de error 409 (correo duplicado) con mensaje inline en tileEmail.

Resultado: Flujo de registro completo sin servidor de correo (verificación pendiente).

Sesión 9.4 — Módulo de Agenda: sesiones de entrenamiento

Fecha: 2026-01-30

Actividades:

- Entidad SesionEntrenamiento con estados ENUM: PROGRAMADA, ACTIVA, FINALIZADA, CANCELADA.
- Endpoints: GET /sesiones (filtrado por semana), POST /sesiones, PUT /sesiones/{id}.
- AgendaActivity.java: vista de tarjetas con semana actual, navegación
- Tarjetas de sesión: hora, lugar, grupo. Fondo rojo para sesiones CANCELADA.

Resultado: Agenda visible para el atleta con semanas navegables.

Sesión 9.5 — Módulo de Asistencia

Fecha: 2026-02-06

Actividades:

- Entidad RegistroAsistencia con estado: PRESENTE, AUSENTE, JUSTIFICADO.
- POST /sesiones/{id}/asistencia — registro con lista de atletas.
- GET /asistencia/historial?atletaId=X — historial individual.
- AsistenciaActivity.java: lista de atletas del grupo con switches de marcación.
- Cálculo de porcentaje de asistencia en el backend.

Resultado: El entrenador puede tomar lista digitalmente desde la app.

diagnóstico:

- AgendaActivity.java: eliminados toasts de diagnóstico de cargarSesiones() y aplicarFiltros(). Error en onFailure/onResponse error → getString(R.string.err_conexion).
- SesionAdapter.java: eliminado setBackgroundColor(RED).
- CrearSesionActivity.java: mensajes onFailure de diagnóstico ("Grupos/...", "Guardar/...", "Cancelar/...") reemplazados por getString(R.string.err_conexion).

Resultado: Las sesiones creadas son visibles en el módulo Agenda con el card teal bien delimitado contra el fondo navy oscuro.

Anexo B — Fragmentos de Código Fuente Representativos

B.1 JwtService.java — Generación y validación de JWT

```
@Service
public class JwtService {
    private final String secretKey = System.getenv("JWT_SECRET");
    private final long expirationMs = 30L * 24 * 60 * 60 * 1000;

    public String generateToken(Usuario usuario) {
        return Jwts.builder()
            .setSubject(usuario.getCorreo())
            .claim("rol", usuario.getRol().name())
            .claim("userId", usuario.getId())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public boolean isTokenValid(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(getSigningKey()).build()
                .parseClaimsJws(token);
            return true;
        } catch (JwtException e) {
            return false;
        }
    }

    private Key getSigningKey() {
        return Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8));
    }
}
```

B.2 FcmService.java — Envío de notificación push con verificación de preferencias

```
@Service
public class FcmService {

    @Async
    public void sendToUser(Usuario usuario, String titulo, String cuerpo,
        TipoNotificacion tipo) {
        if (!aceptaNotificacion(usuario, tipo)) return;
    }
}
```

```

String fcmToken = usuario.getFcmToken();
if (fcmToken == null || fcmToken.isBlank()) return;

Message message = Message.builder()
    .setToken(fcmToken)
    .setNotification(Notification.builder()
        .setTitle(titulo).setBody(cuerpo).build())
    .putData("tipo", tipo.name())
    .build();

try {
    FirebaseMessaging.getInstance().send(message);
} catch (FirebaseMessagingException e) {
    log.error("Error FCM para usuario {}: {}", usuario.getId(), e.getMessage());
}

private boolean aceptaNotificacion(Usuario u, TipoNotificacion tipo) {
    return switch (tipo) {
        case SESION -> !Boolean.FALSE.equals(u.getNotifSesiones());
        case COMPETENCIA -> !Boolean.FALSE.equals(u.getNotifCompetencias());
        case RESULTADO -> !Boolean.FALSE.equals(u.getNotifResultados());
    };
}
}

```

B.3 ApiClient.java — Singleton Retrofit con interceptores JWT y 401

```

public class ApiClient {
    private static final String BASE_URL = "http://45.xxx.xxx.xxx:8080/";
    private static Retrofit retrofit;

    public static Retrofit getInstance(Context context) {
        if (retrofit == null) {
            SessionManager session = new SessionManager(context);

            OkHttpClient client = new OkHttpClient.Builder()
                .addInterceptor(chain -> {
                    Request req = chain.request().newBuilder()
                        .addHeader("Authorization", "Bearer " + session.getToken())
                        .build();
                    return chain.proceed(req);
                })
                .addInterceptor(chain -> {
                    Response resp = chain.proceed(chain.request());
                    if (resp.code() == 401) {
                        session.clearSession();
                        Intent intent = new Intent(context, LoginActivity.class);
                        intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
                            Intent.FLAG_ACTIVITY_CLEAR_TASK);
                        context.startActivity(intent);
                    }
                    return resp;
                })
                .build();

            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
    }
}

```

```

    }
    return retrofit;
}
}

```

B.4 CategoriaSchedulerService.java — Actualización automática de categorías

```

@Service
public class CategoriaSchedulerService {

    private final UsuarioRepository usuarioRepo;
    private final NotificacionService notifService;

    @Scheduled(cron = "0 0 1 * * ?")
    public void actualizarCategorias() {
        LocalDate hoy = LocalDate.now();
        List<Usuario> atletas = usuarioRepo.findByRolAndActivo(Rol.ATLETA, true);

        for (Usuario atleta : atletas) {
            if (atleta.getFechaNacimiento() == null) continue;
            int edad = Period.between(atleta.getFechaNacimiento(), hoy).getYears();
            CategoriaEtaria nueva = CategoriaEtaria.fromEdad(edad);

            if (nueva != atleta.getCategoria()) {
                atleta.setCategoria(nueva);
                usuarioRepo.save(atleta);
                notifService.enviar(atleta,
                    "Categoría actualizada",
                    "Ahora compites en la categoría " + nueva.getDisplayName(),
                    TipoNotificacion.SESION);
            }
        }
    }
}

```

B.5 EvolucionMarcasActivity.java — Gráfica MPAndroidChart con Cubic Bezier

```

private void renderizarGrafica(List<MarcaResponse> marcas) {
    List<Entry> entries = new ArrayList<>();
    for (int i = 0; i < marcas.size(); i++) {
        entries.add(new Entry(i, (float) marcas.get(i).getResultado()));
    }

    LineDataSet dataSet = new LineDataSet(entries, "Marcas - " + disciplinaActual);
    dataSet.setMode(LineDataSet.Mode.CUBIC_BEZIER);
    dataSet.setColor(getColor(R.color.colorPrimary));
    dataSet.setCircleColor(getColor(R.color.colorPrimary));
    dataSet.setLineWidth(2.5f);
    dataSet.setCircleRadius(4f);
    dataSet.setDrawFilled(true);
    dataSet.setFillColor(getColor(R.color.colorPrimary));
    dataSet.setFillAlpha(50);
    dataSet.setValueTextColor(getColor(R.color.colorTextPrimary));
    dataSet.setValueTextSize(10f);

    grafica.setData(new LineData(dataSet));
    grafica.getDescription().setEnabled(false);
    grafica.getLegend().setEnabled(false);
    grafica.animateXY(800, 800);
}

```

```

    grafica.invalidate();
}

```

Anexo C — Entrevista de Levantamiento de Requisitos

Entrevista realizada con el entrenador del Club Atlético Santa Cruz de la Sierra.

Modalidad: presencial, duración aproximada 45 minutos.

Fecha: diciembre de 2025.

Entrevistador: Buenos días. Para comenzar, ¿podría describir cómo es el Club Atlético Santa Cruz actualmente?

Entrenador: Tenemos aproximadamente 45 atletas en cuatro grupos: Pre-Infantil de 8 a 10 años, Infantil de 11 a 13, Juvenil de 14 a 17 y Mayores. Practicamos velocidad, salto largo, lanzamiento de bala y gimnasia artística.

Entrevistador: ¿Cómo manejan actualmente los horarios de entrenamiento?

Entrenador: Por WhatsApp. Tengo un grupo para cada categoría. Cuando hay algún cambio lo pongo ahí, pero muchos chicos no leen a tiempo y llegan al entrenamiento o no llegan y no sabíamos que estaba cancelado.

Entrevistador: ¿Y el control de asistencia?

Entrenador: Tengo una planilla en papel. Al final del mes la paso a Excel pero a veces se me olvida o se pierde la hoja. No tengo un registro histórico confiable.

Entrevistador: ¿Cómo registran las marcas y tiempos?

Entrenador: Las anoto en una libreta. A veces la foto del cronómetro por WhatsApp. El chico no sabe cuánto mejoró a menos que yo le diga.

Entrevistador: ¿Realizan competencias?

Entrenador: Sí, la Federación nos convoca. Yo llamo a los atletas por WhatsApp para ver quién puede ir. Los resultados los tengo en fotos del cronómetro o en papel.

Entrevistador: Sobre los atletas menores, ¿tienen algún protocolo de privacidad?

Entrenador: No realmente. La información circula en el grupo de WhatsApp con todos los padres y atletas mezclados.

Entrevistador: Si pudiera tener una sola cosa en una app, ¿qué sería lo más urgente?

Entrenador: Que los chicos sepan sus entrenamientos sin que yo tenga que mandarles mensaje cada vez. Y que me avise automáticamente cuando hay un cambio de horario.

Entrevistador: ¿Qué tanto tiempo le toma la gestión administrativa actualmente?

Entrenador: Unas 2 horas semanales solo en mensajes y listas. Preferiría usar ese tiempo en planificar el entrenamiento.

Entrevistador: ¿Tienen acceso a internet en la cancha?

Entrenador: No siempre. A veces no hay señal en la pista municipal. Sería bueno que funcione aunque sea sin internet para tomar la asistencia.

Entrevistador: ¿Los padres estarían dispuestos a usar la app?

Entrenador: Los que tienen hijos menores sí, seguro. Siempre preguntan horarios y si el chico asistió.

Entrevistador: ¿Alguna funcionalidad que definitivamente NO quiera?

Entrenador: No quiero nada complicado. Una cosa a la vez. Si el chico puede ver su agenda, su asistencia y sus marcas, con eso ya es un avance enorme.

Conclusiones del levantamiento:

Los 5 dolores principales identificados:

56. WhatsApp como canal de comunicación operativa → no llega a tiempo, no hay confirmación de lectura.

57. Planillas de asistencia en papel → pérdida de información histórica.

58. **Libretas de marcas** → el atleta no tiene acceso a su propia evolución.

59. **Gestión de competencias por mensajes privados** → sin trazabilidad ni confirmación formal.

60. **Datos de menores sin control de acceso** → circulan en grupos abiertos de WhatsApp.

Anexo E — Glosario de Términos

Término	Definición
APK	Android Package Kit. Formato de archivo de instalación para aplicaciones Android.
API REST	Interfaz de programación que expone recursos mediante URLs y métodos HTTP estándar (GET, POST, PUT, DELETE).
BCrypt	Función de hash de contraseñas con sal incorporada, diseñada para ser computacionalmente costosa y resistente a ataques de fuerza bruta.
CI/CD	Integración Continua / Despliegue Continuo. Prácticas de ingeniería que automatizan la compilación, prueba y despliegue de software.
Coolify	Plataforma open source de orquestación de contenedores autoalojable, alternativa a Heroku/Vercel para VPS propios.
DAO	Data Access Object. Patrón de diseño que abstrae el acceso a la fuente de datos. En Spring, implementado por los <code>Repository</code> .
DTO	Data Transfer Object. Objeto simple que transporta datos entre capas sin lógica de negocio.
FCM	Firebase Cloud Messaging. Servicio de Google para el envío de notificaciones push a dispositivos móviles.
Gradle	Sistema de automatización de construcción (build) utilizado en proyectos Java, Kotlin y Android.
JWT	JSON Web Token. Estándar RFC 7519 para transmitir información firmada digitalmente entre partes.
Hibernate	Framework ORM que mapea clases Java a tablas de base de datos relacional.
Material Design 3	Sistema de diseño visual de Google para aplicaciones Android, iOS y web.
MPAndroidChart	Librería open source de gráficas para Android (línea, barras, torta, radar, entre otras).
ORM	Object-Relational Mapping. Técnica de mapeo entre objetos de programación y tablas relacionales.
OkHttp	Cliente HTTP de alto rendimiento para aplicaciones Android y Java, utilizado por Retrofit.
PostgreSQL	Sistema de gestión de bases de datos relacionales open source, conocido por su robustez y soporte de tipos avanzados.
Push Notification	Mensaje enviado desde un servidor a un dispositivo

	cliente sin que este lo solicite. En Android, via FCM.
Retrofit	Librería Android que convierte interfaces Java anotadas en llamadas HTTP a una API REST.
Scrum	Marco de trabajo ágil para el desarrollo de software, basado en sprints de duración fija, backlog priorizado y revisiones iterativas.
SharedPreferences	Mecanismo de almacenamiento de datos clave-valor en Android, persistente entre sesiones de la app.
Spring Boot	Framework Java que simplifica la creación de aplicaciones Spring mediante autoconfiguración y servidor embebido.
Spring Security	Módulo de Spring para autenticación, autorización y protección contra ataques comunes.
VPS	Virtual Private Server. Servidor virtual alojado en la nube con acceso root, utilizado para el despliegue del backend.
ENUM	Tipo de dato que restringe los valores posibles a un conjunto predefinido. En SQL y Java, usado para estados y categorías.
Webhook	Notificación HTTP enviada automáticamente por un sistema a otro cuando ocurre un evento específico.
Deep Link	URL especial que abre una pantalla específica de una app móvil desde el navegador o un correo electrónico.
Marca personal	En atletismo, el mejor resultado obtenido por un atleta en una disciplina a lo largo de toda su carrera.
Categoría etaria	Clasificación de atletas según su edad: Pre-Infantil (8-10), Infantil (11-13), Juvenil (14-17), Mayores (+18).